
Software Requirements Specification

for

Movie Ticket Booking System

Platform

Version 1.0 Approved

Prepared by Barath N

15-10-2026

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Document Conventions.....	4
1.3 Intended Audience and Reading Suggestions.....	4
1.4 Product Scope.....	4
1.5 References.....	5
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Functions.....	5
2.3 User Classes and Characteristics.....	6
2.3.1 User Class 1: Administrator.....	6
2.3.2 User Class 2: Theater Manager.....	8
2.3.3 User Class 3: Customer.....	9
2.4 Operating Environment.....	11
2.5 Design and Implementation Constraints.....	11
2.6 User Documentation.....	12
2.7 Assumptions and Dependencies.....	12
3. External Interface Requirements.....	12
3.1 User Interfaces.....	12
3.2 Hardware Interfaces.....	12
3.3 Software Interfaces.....	12
3.4 Communications Interfaces.....	13
4. System Features.....	13
4.1 User Authentication and Role-Based Access Control.....	13
4.2 Movie Catalog & Showtime Scheduling.....	13
4.3 Interactive Seat Selection & Real-Time Availability.....	14
4.4 Ticket Booking & Checkout Processing.....	14
4.5 Digital Ticketing & Booking History.....	15
4.6 Theater Manager Sales & Financial Reporting.....	15
4.7 System Administration & Manager Provisioning.....	16
5. Other Nonfunctional Requirements.....	16
5.1 Performance Requirements.....	16
5.2 Safety Requirements.....	17
5.3 Security Requirements.....	17
5.4 Software Quality Attributes.....	17
5.5 Business Rules.....	17
6. Other Requirements.....	18
6.1 Database Requirements.....	18
6.2 Internationalization Requirements.....	18

6.3 Legal and Regulatory Requirements.....	18
6.4 Accessibility Requirements.....	18
Appendix A: Glossary.....	19
Appendix B: Analysis Models.....	20
Appendix C: Source Code.....	31
Appendix D: To Be Determined List.....	66

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements of the Movie Ticket Booking System (MTBS / CinePass), Version 1.0 Approved. The MTBS is a full-stack, standalone web-based ticketing platform designed to replace legacy manual box-office counter operations, physical paper ticket registers, and uncoordinated movie scheduling with a unified, high-performance digital cinema management solution. This document establishes the official engineering baseline for system architecture, database schema design, component interaction, testing, verification, and operational deployment.

1.2 Document Conventions

This document adheres to the IEEE Std 830-1998 Recommended Practice for Software Requirements Specifications:

- SHALL denotes mandatory requirements; SHOULD denotes highly recommended features.
- Functional requirements are tagged as FR-XX and non-functional requirements as NFR-XX.
- Section headings follow a standardized hierarchical decimal numbering system.
- Specific user roles—Administrator, Theater Manager, and Customer—are explicitly defined throughout.
- All monetary transactions are specified in Indian Rupees (INR / ₹) with UTC timestamps.

1.3 Intended Audience and Reading Suggestions

The intended audience for this document includes:

- Software Developers & Engineers: Design and implement the JSP servlets, DAO layers, and UI components (Sections 2, 3, 4, Appendix B).
- Project Managers & Quality Assurance: Track feature milestones and design validation test cases (Sections 1, 2, 4, 5).
- Cinema Theater Executives: Evaluate administrative provisioning and revenue analytics capabilities (Sections 1, 2, 4).
- Database Administrators: Inspect relational schema integrity and 3NF normalization (Section 6.1, Appendix B.2).

1.4 Product Scope

The Movie Ticket Booking System (MTBS / CinePass) is an end-to-end, enterprise-grade web application tailored for cinema multiplexes, independent movie theaters, and film distribution networks. Key capabilities of MTBS encompass:

- Role-Based Access Control: Specialized authentication and privilege isolation across Customers, Theater Managers, and Administrators.
- Dynamic Movie Catalog Management: Provisioning of movie releases with genres, durations, languages, and storylines.

- Multiplex Showtime Scheduling: Mapping cinema screens (e.g., Screen 1 IMAX) to show dates, showtimes, and ticket pricing.
- Interactive Graphical Seat Selection: Real-time visual seat map indicating available, locked, and booked seats.
- Transactional Booking Engine: Atomic ACID-compliant seat reservations with simulated payment checkout (UPI, Cards, Net Banking).
- Instant Digital Ticketing: Auto-generation of confirmed ticket passes with unique Booking IDs and printable admission barcodes.
- Customer Booking History & Self-Service Cancellation: Real-time booking tracking and instant seat release.
- Managerial Sales Reporting & Revenue Audit: Real-time tracking of total box-office revenue, confirmed bookings, and transaction audits.
- System Administration & Staff Provisioning: User directory audit, account removal, and manager onboarding.

1.5 References

1. IEEE Std 830-1998, 'IEEE Recommended Practice for Software Requirements Specifications', IEEE, 1998.
2. Sommerville, Ian, 'Software Engineering', 10th Edition, Pearson Education, 2015.
3. Pressman, Roger S., 'Software Engineering: A Practitioner's Approach', 8th Edition, McGraw-Hill, 2014.
4. Oracle Corporation, 'Java Servlet Specification 4.0 / Jakarta Servlet API', Oracle / Eclipse Foundation.
5. Oracle Corporation, 'MySQL 8.0 Reference Manual', <https://dev.mysql.com/doc/>.
6. Apache Software Foundation, 'Apache Tomcat 9.0 Architecture & Documentation', <https://tomcat.apache.org/>.

2. Overall Description

2.1 Product Perspective

The MTBS operates as an autonomous, self-contained Dynamic Web Application built on Java EE (Servlets & JSP) and MySQL. It completely modernizes traditional box-office queues by shifting ticket selection, seat allocation, and payment into an always-accessible digital self-service channel. The system adopts a standard three-tier architecture comprising a presentation client (Web Browser rendering responsive HTML5/CSS3 glassmorphism interfaces), an application tier (Apache Tomcat 9.0 container running Java Servlets and Session managers), and an enterprise data persistence tier (MySQL 8.0).

2.2 Product Functions

The major business functions provided by MTBS include:

1. User Registration & Secure Authentication: Self-service customer sign-up with encrypted credential validation.
2. Movies Catalog Administration: Adding, categorizing, and deleting movie titles with metadata.

3. Showtime & Screen Hall Allocation: Scheduling screening dates, times, screen halls, and base ticket prices.
4. Graphical Seat Selection & Dynamic Availability: Real-time interactive seat grid rendering booked vs available seats.
5. Atomic Booking & Concurrency Control: Two-phase database transactions preventing double-booking of seats.
6. Multi-Channel Payment Simulation: Integrated checkout supporting UPI, Credit/Debit Cards, and Net Banking.
7. Digital Ticket Issuance & Printing: Issuing digital admission passes complete with barcode and booking reference.
8. Self-Service Booking Cancellation: Customer-initiated booking cancellation with automated seat status reversion.
9. Manager Sales Analytics & Audit Reports: Revenue calculation, ticket volume tracking, and financial payment audit.
10. System Administrator Control Center: User directory oversight, role auditing, and dedicated manager provisioning.

2.3 User Classes and Characteristics

The MTBS platform serves three distinct user classes: Administrator, Theater Manager, and Customer. The following subsections present the comprehensive use case specifications for each actor class, followed by dedicated reserved spaces for the corresponding UML Use Case Diagrams.

2.3.1 User Class 1: Administrator

The System Administrator holds the highest level of administrative privileges within the MTBS ecosystem. The Administrator is responsible for system governance, user directory auditing, account removals, and provisioning theater manager accounts.

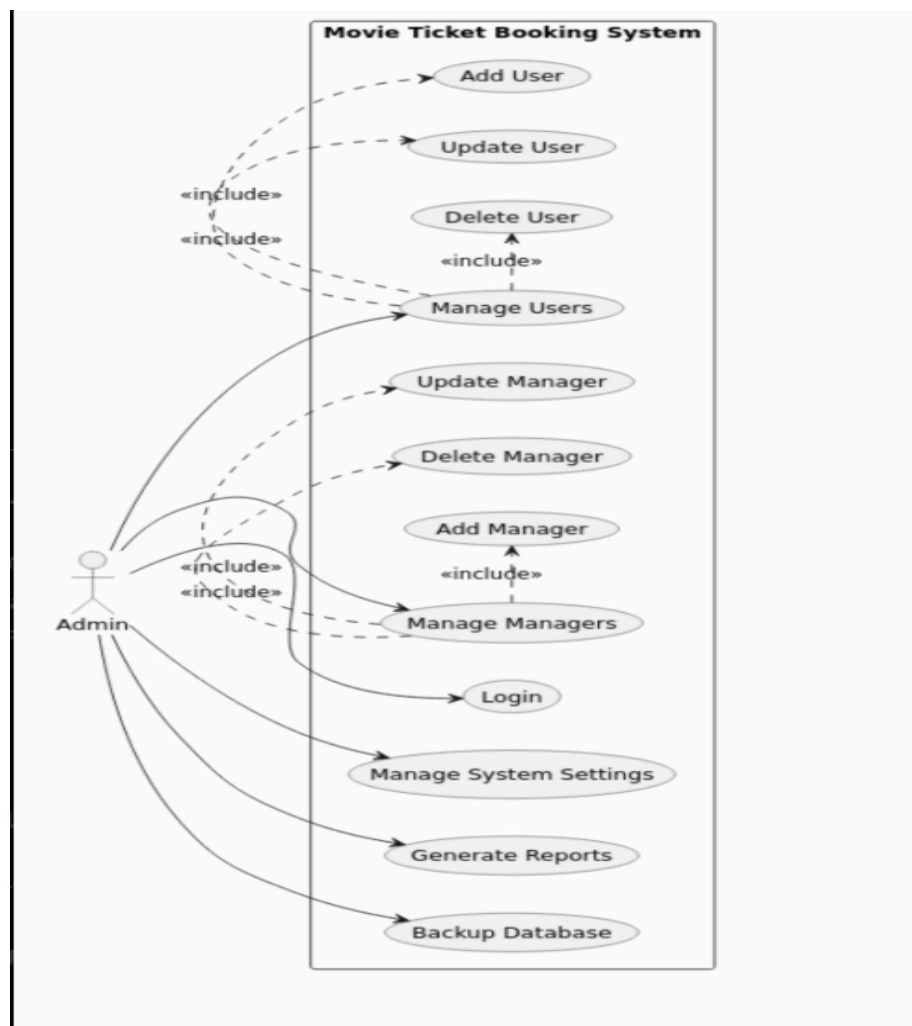


Figure 2.1 Admin use case diagram

Use Case: Manage Users & Audit Directory**Primary Actor:** Administrator**Secondary Actor:** System Database**Precondition:**

- Administrator is authenticated and logged in with role 'ADMIN'.
- System database server is active and accessible.

Postcondition:

- User account directory is audited.
- User accounts are created, updated, disabled, or removed from the database.

Trigger:

- Administrator selects 'Manage Users' or opens System User Directory.

Main Access Scenario:

- Administrator logs into the system.

- Opens the user management module.
- Searches for an existing user or creates a new user.
- Adds, updates, or removes user information.
- Assigns appropriate user roles (ADMIN, MANAGER, CUSTOMER).
- Saves the changes.
- System updates the user database.

Exception Scenario:

- Duplicate user ID or email address.
- Self-deletion attempt by logged-in Administrator.
- Required information is missing.
- Database update fails.

2.3.2 User Class 2: Theater Manager

Theater Managers supervise cinema multiplex operations. They manage the movies catalog, schedule screening slots across theater halls, assign ticket prices, and monitor sales performance and revenue analytics through dedicated reports.

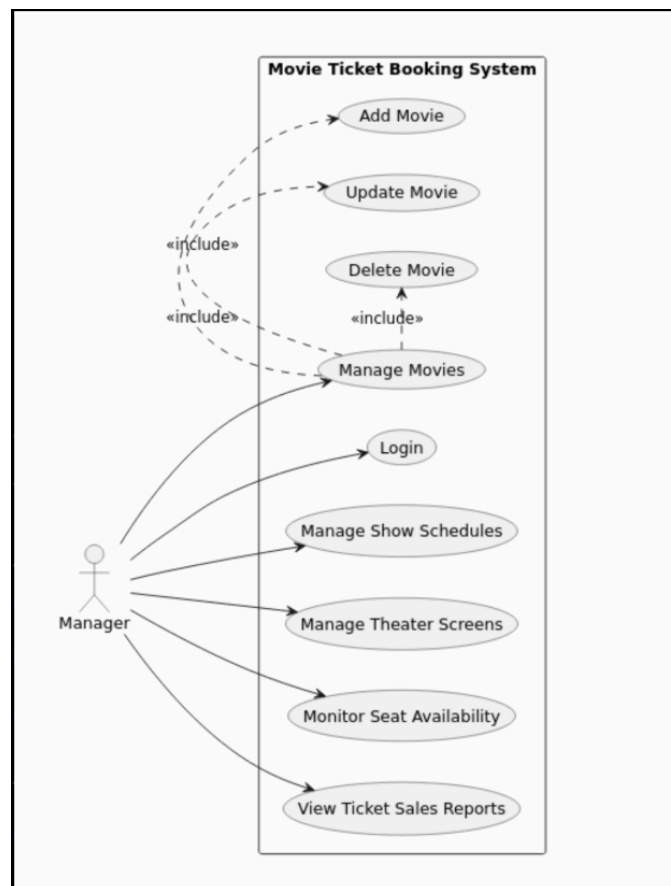


Figure 2.2 Manager use case diagram

Use Case: Manage Movies & Schedule Showtimes**Primary Actor:** Theater Manager**Secondary Actor:** System Database**Precondition:**

- Theater Manager is authenticated and logged in with role 'MANAGER'.
- At least one active screen hall exists in the multiplex.

Postcondition:

- Movie title is registered in the theater catalog.
- Showtime slot is saved into database with date, time, screen hall, and ticket price.

Trigger:

- Manager selects 'Add Movie' or 'Manage Show Schedules'.

Main Access Scenario:

- Manager logs into the system.
- Navigates to movie catalog management portal.
- Inputs movie details (title, genre, language, duration, description).
- Submits movie record to catalog.
- Selects screen hall, screening show date, time slot, and ticket price.
- Confirms showtime allocation.
- System validates input and updates active cinema schedule.

Exception Scenario:

- Invalid duration or missing required movie fields.
- Showtime schedule conflict on selected screen hall.
- Database commit fails.

2.3.3 User Class 3: Customer

Customers are public moviegoers who access MTBS to browse current theatrical releases, select preferred seats in theater screens, execute simulated payments, view digital tickets, and manage their booking history with cancellation options.

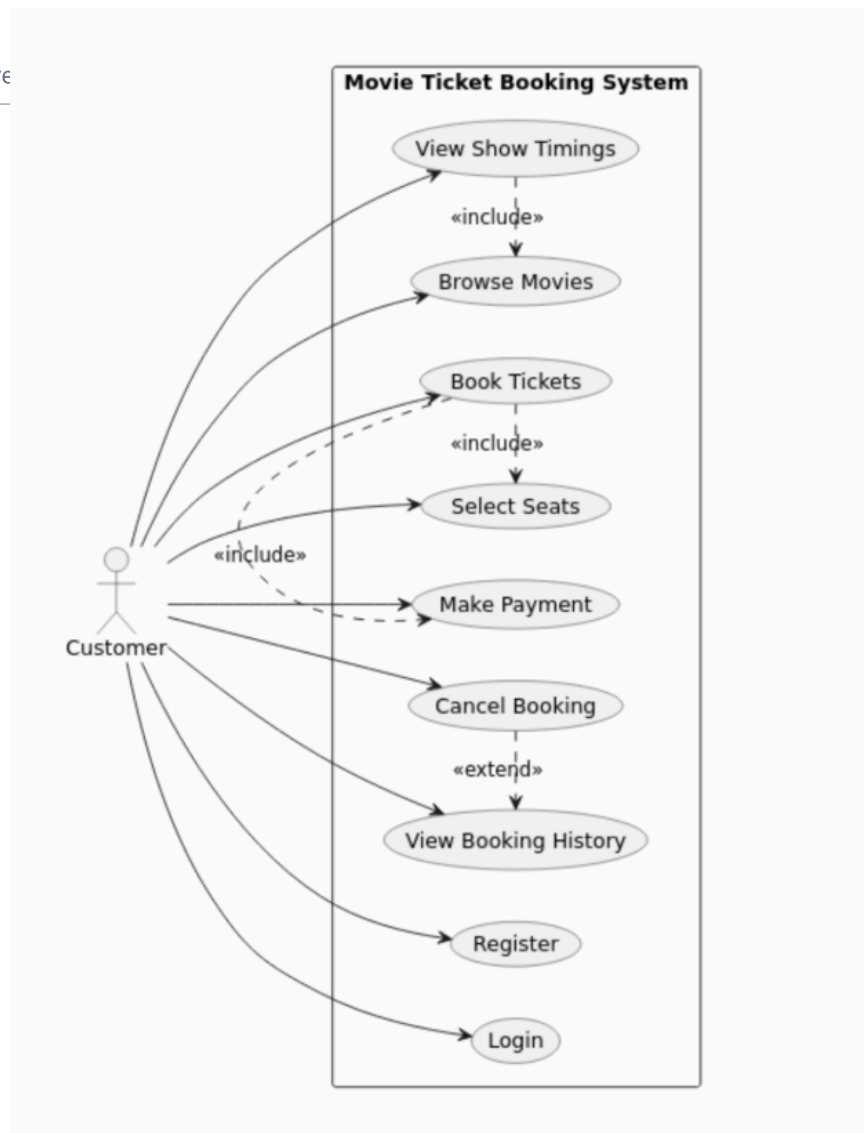


Figure 2.3 Customer use case diagram

Use Case: Book Tickets & Select Seats**Primary Actor:** Customer / Member**Secondary Actor:** Payment Gateway Simulation, System Database**Precondition:**

- Customer is registered and logged into the system.
- Selected movie showtime exists and has available seats.

Postcondition:

- Booking record is created and confirmed.
- Selected seats are marked as BOOKED.
- Borrowing and booking history is updated.
- Digital ticket receipt with entrance barcode is generated.

Trigger:

- Customer selects 'Book Tickets' or 'Select Seats' option.

Main Access Scenario:

- Customer logs into the system.
- Searches for and selects a movie title from active catalog.
- Chooses preferred screening showtime and screen hall.
- Selects available seats on interactive graphical seat map.
- Clicks 'Proceed to Checkout'.
- Selects payment method (UPI, Card, Net Banking) and submits payment.
- System verifies transaction, locks seats, and updates booking history.
- System generates digital admission ticket.

Exception Scenario:

- Selected seat already booked by another user during checkout.
- Payment authorization fails or times out.
- Customer has exceeded maximum ticket booking limit.

2.4 Operating Environment

The MTBS platform operates across the following production and development configurations:

- Client Side: Google Chrome v100+, Mozilla Firefox v100+, Microsoft Edge v100+, or Safari v15+ on Windows 10/11, macOS, Android, or iOS devices with minimum screen resolution of 1280x720 (responsive down to 360x640 mobile viewports).
- Web & Application Server: Apache Tomcat 9.0+ executing Java SE 17 or Java SE 21 on Windows Server or Linux (Ubuntu 22.04 LTS). Standard JVM memory allocation: minimum 2 GB RAM, 2-core CPU.
- Database Management System: MySQL Server 8.0+ running on localhost port 3306 with UTF-8 character encoding and InnoDB transactional storage engine.
- Network: Standard HTTP/1.1 or secure HTTPS (TLS 1.2+) network connectivity over TCP/IP.

2.5 Design and Implementation Constraints

- Architectural Framework: Strict Java Enterprise Dynamic Web Project standard using Servlets, JavaServer Pages (JSP), and standard JDBC data access objects (DAO).
- Session Governance: Server-side HTTP session tracking for role verification across all protected routes.
- UI Design Language: Modern CSS3 styling utilizing glassmorphism visual effects, gradient accents, Flexbox, and CSS Grid without external heavyweight front-end frameworks.
- Database Referential Integrity: Enforcement of foreign key constraints across users, movies, screens, shows, seats, bookings, booking_seats, and payments.
- Academic & Standalone Operability: Complete portability without cloud dependencies, allowing seamless offline demonstration.

2.6 User Documentation

Documentation supplied with MTBS includes:

- Comprehensive Software Requirements Specification (this IEEE 830 document).

- Database Schema SQL Script (database.sql) with table initialization and sample cinema records.
- System Setup & Execution Guide describing Tomcat deployment and MySQL connection parameters.

2.7 Assumptions and Dependencies

- Assumptions: The cinema multiplex operates fixed seating layouts per screen hall; users have standard modern web browsers with JavaScript enabled.
- Dependencies: Java Runtime Environment (JRE 17+), Apache Tomcat 9.0 servlet container, MySQL Connector/J JDBC driver library (mysql-connector-j-26.7.0.jar).

3. External Interface Requirements

3.1 User Interfaces

- Customer Portal: Clean, responsive interface featuring hero banners, movie catalog grids with genre and duration chips, interactive visual cinema seat maps, checkout forms, and printable digital ticket stubs.
- Theater Manager Control Center: Operational dashboard displaying movie scheduling tables, add movie forms, screening slot configurators, and real-time box office sales analytics.
- Administrator Console: Dedicated administrative dashboard providing system-wide user directory tables, manager provisioning forms, and role governance controls.

3.2 Hardware Interfaces

Standard client devices (PCs, laptops, tablets, mobile devices) communicating over standard network adapters (Ethernet / Wi-Fi). No specialized proprietary hardware is required.

3.3 Software Interfaces

Software Component	Version	Interface & Interaction Description
MySQL Database	8.0+	Stores all relational cinema entities; connected via JDBC URL on port 3306.
Apache Tomcat	9.0.x	Servlet container executing Java Servlets, managing thread pools, and compiling JSPs.
Java Servlet API	javax.servlet 4.0	Core API for HTTP request dispatching, session handling, and response routing.
MySQL Connector/J	26.7.0 / 8.0+	Type 4 JDBC driver facilitating high-throughput connection pooling.

3.4 Communications Interfaces

- Protocol: HTTP/1.1 and HTTPS (TLS 1.2+) over standard TCP/IP ports 8080 / 443.
- Content Types: text/html, application/x-www-form-urlencoded, and multipart/form-data.

4. System Features

4.1 User Authentication and Role-Based Access Control

Enforces strict session-based authentication and role segregation across Customer, Manager, and Admin roles.

Req ID	Functional Requirement Description
FR-01	The system SHALL provide unified login authentication verifying user credentials against the MySQL database.
FR-02	The system SHALL maintain active user sessions via javax.servlet.http.HttpSession holding userId, name, and role.
FR-03	The system SHALL enforce role-based access control, preventing customers from accessing manager or admin endpoints.
FR-04	The system SHALL provide public self-service registration for customer accounts.
FR-05	The system SHALL provide secure session invalidation and logout.

4.2 Movie Catalog & Showtime Scheduling

Empowers theater managers to manage films and showtime schedules.

Req ID	Functional Requirement Description
FR-06	The system SHALL allow managers to register new movie titles with genre, language, duration, and synopsis.
FR-07	The system SHALL automatically schedule a default screening slot upon new movie creation.
FR-08	The system SHALL display all active movies in the customer catalog with duration and language chips.

FR-09	The system SHALL allow managers to assign movie screenings to screen halls with specific dates and showtimes.
FR-10	The system SHALL allow managers to delete expired movie titles from the catalog.

4.3 Interactive Seat Selection & Real-Time Availability

Renders the graphical cinema seating layout and reflects booked statuses dynamically.

Req ID	Functional Requirement Description
FR-11	The system SHALL render an interactive seating grid with distinct seat numbers (e.g. A1-A4, B1-B4).
FR-12	The system SHALL query the seats table and disable already-booked seats with visual blocked indicators.
FR-13	The system SHALL dynamically calculate total payable amount via client-side script as seats are selected.
FR-14	The system SHALL prevent booking submission when zero seats are selected.

4.4 Ticket Booking & Checkout Processing

Executes two-phase transactional reservations and payment processing.

Req ID	Functional Requirement Description
FR-15	The system SHALL execute seat booking within an atomic database transaction with rollback capability.
FR-16	The system SHALL update seat status from 'AVAILABLE' to 'BOOKED' atomically.
FR-17	The system SHALL generate a unique Booking ID and forward the transaction to the payment gateway.
FR-18	The system SHALL support payment simulation across UPI, Credit/Debit Cards, and Net Banking.
FR-19	The system SHALL store successful payment records in the payments table linked to the

	Booking ID.
--	-------------

4.5 Digital Ticketing & Booking History

Generates confirmed digital admission tickets and enables customer booking tracking.

Req ID	Functional Requirement Description
FR-20	The system SHALL generate a digital ticket receipt displaying movie title, showtime, screen, and barcode.
FR-21	The system SHALL provide a browser print action for physical ticket printing.
FR-22	The system SHALL maintain a comprehensive booking history accessible to logged-in customers.
FR-23	The system SHALL allow customers to cancel confirmed bookings, automatically reverting seats to 'AVAILABLE'.

4.6 Theater Manager Sales & Financial Reporting

Delivers financial visibility and ticket sales analytics exclusively to Theater Managers.

Req ID	Functional Requirement Description
FR-24	The system SHALL provide a dedicated Sales Report portal accessible exclusively to role 'MANAGER'.
FR-25	The system SHALL calculate real-time box office gross revenue from confirmed bookings.
FR-26	The system SHALL display an audit log of recent transactions with payment methods and timestamps.
FR-27	The system SHALL restrict Administrator and Customer access from the sales report.

4.7 System Administration & Manager Provisioning

Provides governance tools for platform administrators.

Req ID	Functional Requirement Description
FR-28	The system SHALL provide an administrative User Directory listing all registered accounts with role badges.
FR-29	The system SHALL enable administrators to delete obsolete accounts with self-deletion protection.
FR-30	The system SHALL provide a dedicated manager provisioning form creating accounts with role 'MANAGER'.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

NFR ID	Performance Requirement Description
NFR-01	The system SHALL support at least 250 concurrent user sessions without performance degradation.
NFR-02	All page requests SHALL load within 1.5 seconds on standard broadband connections.
NFR-03	Database read queries for movies and seat availability SHALL execute within 150 milliseconds.
NFR-04	The transactional booking submission SHALL complete within 500 milliseconds.

5.2 Safety Requirements

NFR ID	Safety Requirement Description
NFR-05	Database transactions SHALL implement automatic rollback upon any SQL exception to maintain zero data corruption.
NFR-06	All user-facing errors SHALL be presented via sanitized alert banners without exposing stack traces or database schema details.

5.3 Security Requirements

NFR ID	Security Requirement Description
NFR-07	Role-based access checks SHALL be verified at the beginning of every restricted JSP and Servlet.
NFR-08	All SQL interactions SHALL utilize PreparedStatements with parameterized queries to prevent SQL Injection attacks.
NFR-09	Administrators SHALL be prohibited from deleting their own active administrator accounts.

5.4 Software Quality Attributes

- Availability: Target 99.8% uptime during active cinema operating hours (8:00 AM – 1:00 AM).
- Usability: Intuitive glassmorphism interface enabling customers to complete seat bookings in under 4 clicks.
- Maintainability: Modular MVC structure separating controllers (Servlets), presentation (JSP), and data access (DAO).
- Reliability: ACID database transaction guarantees eliminating duplicate ticket issuance.

5.5 Business Rules

- BR-01: A seat can only belong to one confirmed booking at any given time.
- BR-02: Cancelling a booking instantly reverts all associated seats to 'AVAILABLE' status.
- BR-03: Sales revenue reports are strictly reserved for Theater Managers.
- BR-04: Only Administrators are authorized to provision new Theater Manager accounts.

6. Other Requirements

6.1 Database Requirements

The database schema shall adhere to Third Normal Form (3NF) and enforce foreign key referential integrity across all tables. Timestamps shall follow UTC format.

6.2 Internationalization Requirements

The system shall encode all text using UTF-8, supporting multi-language movie titles, descriptions, and user names across regional cinema distributions.

6.3 Legal and Regulatory Requirements

The platform shall maintain payment and transaction audit records for a minimum statutory period in accordance with digital commerce regulations.

6.4 Accessibility Requirements

The user interface shall comply with WCAG 2.1 Level AA standards, featuring high-contrast text, legible typography, and keyboard accessibility.

Appendix A: Glossary

Term / Acronym	Definition & Context
MTBS	Movie Ticket Booking System - the web application platform specified in this document.
CinePass	The official commercial brand name for the MTBS application suite.
JSP	JavaServer Pages - server-side technology for generating dynamic web pages.
Servlet	Java classes that respond to network requests, forming the controller layer in MTBS.
DAO	Data Access Object - software pattern abstracting database communication.
ACID	Atomicity, Consistency, Isolation, Durability - database transaction properties enforced by MTBS.
RBAC	Role-Based Access Control - security framework restricting access based on roles (Customer, Manager, Admin).
SLA	Service Level Agreement - performance commitments for response times and availability.
SRS	Software Requirements Specification - this formal technical document.
UPI	Unified Payments Interface - real-time payment protocol simulated in checkout.
3NF	Third Normal Form - relational database normalization standard applied in MTBS schema.

Appendix B: Analysis Models

B.1 System Architecture Overview

The MTBS platform implements a robust Three-Tier Enterprise Architecture separating presentation, business logic, and data persistence:

1. Presentation Tier: Responsive HTML5/CSS3 glassmorphism web interfaces executing inside standard client web browsers.
2. Application Tier: Apache Tomcat 9.0 servlet container executing Java Servlets, managing HTTP sessions, and enforcing business rules.

B.2 Key Database Entities

The relational schema of MTBS is normalized to 3NF and comprises eight primary entities:

Entity Name	Key Attributes	Description & Referential Purpose
users	user_id, name, email, password, phone, role	Stores user identities and roles (CUSTOMER, MANAGER, ADMIN).
movies	movie_id, title, genre, language, duration, description	Stores theatrical movie releases and metadata.
screens	screen_id, screen_name, total_seats	Defines multiplex screening halls and capacity.
shows	show_id, movie_id, screen_id, show_date, show_time, price	Maps movies to screens, dates, times, and ticket prices.
seats	seat_id, screen_id, seat_number, status	Tracks individual seats and availability (AVAILABLE / BOOKED).
bookings	booking_id, user_id, show_id, booking_date, total_amount, status	Records customer ticket orders (CONFIRMED / CANCELLED).
booking_seats	booking_id, seat_id	Associates booked seats with specific booking orders (Many-to-Many).
payments	payment_id, booking_id, amount, payment_method, payment_status, payment_date	Records financial checkout audit transactions.

B.3 Class Diagram

B.3.1 Overview of Domain Model

The class diagram represents the object-oriented structure of MTBS. The core User model forms the foundation for specialized roles (Customer, Manager, Administrator). Supporting domain entities include Movie, Screen, Show, Seat, Booking, and Payment, each encapsulated with business operations and database persistence methods.

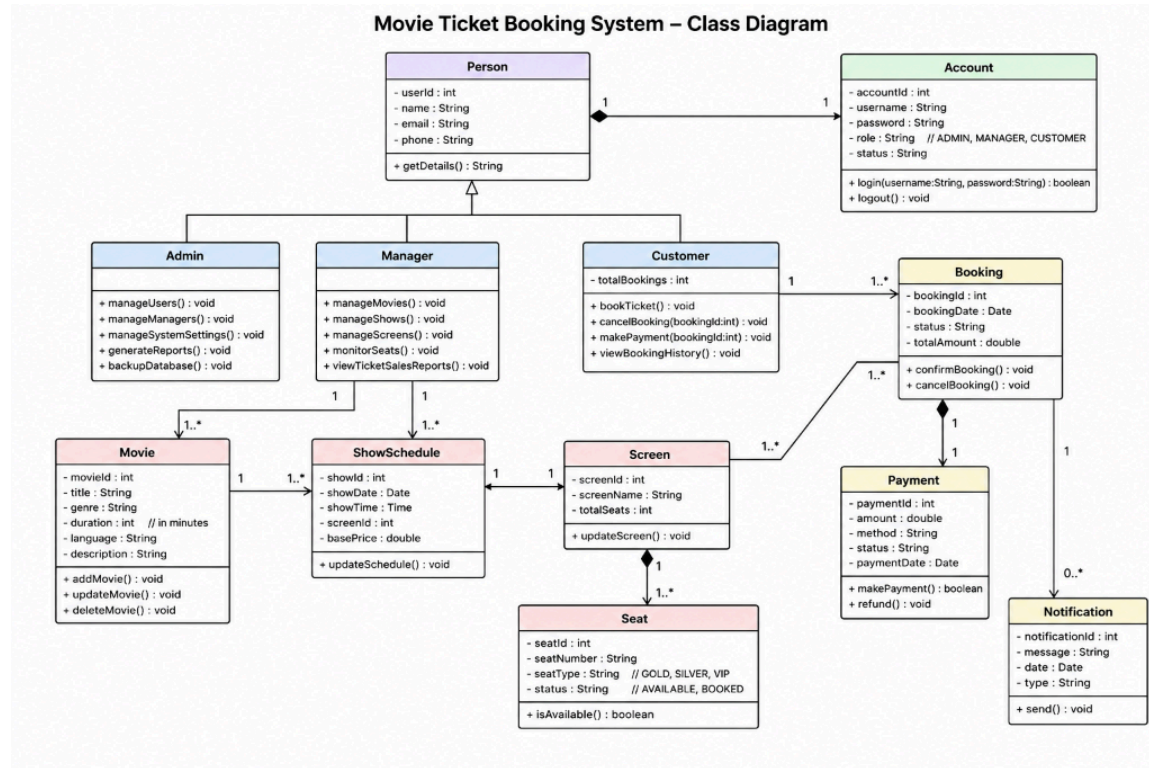


Figure B.1 Movie Ticket Booking System Class Diagram

B.3.2 Class Dictionary and Data Definitions

Class Name	Responsibility	Key Attributes	Core Methods / Operations
User	Base identity model holding common credentials and role definitions.	userId, name, email, password, phone, role	login(), logout(), getRole()
Customer	Represents public moviegoers purchasing tickets and managing	userId, bookingHistoryList	browseMovies(), selectSeats(), checkout(), cancelBooking()

	reservations.		
TheaterManager	Represents theater staff managing screening schedules and catalogs.	userId, managedScreenId	addMovie(), deleteMovie(), scheduleShow(), viewSalesReport()
Administrator	Represents platform administrator managing user accounts and managers.	userId, adminPrivileges	manageUserDirectory(), removeUser(), provisionManager()
Movie	Encapsulates movie release information in catalog.	movieId, title, genre, language, duration, description	getDetails(), updateMovie()
Show	Represents a scheduled screening of a movie in a specific hall.	showId, movieId, screenId, showDate, showTime, price	getSchedule(), checkAvailability()
Seat	Represents an individual physical seat inside a screen hall.	seatId, screenId, seatNumber, status	lockSeat(), bookSeat(), releaseSeat()
Booking	Encapsulates a confirmed movie ticket order.	bookingId, userId, showId, bookingDate, totalAmount, status	confirmBooking(), cancelOrder(), calculateTotal()
Payment	Records transaction payment gateway settlement.	paymentId, bookingId, amount, paymentMethod, status, timestamp	processPayment(), verifyTransaction()
DBConnection	Provides shared JDBC database connectivity via connection pooling.	URL, USER, PASSWORD	getConnection()

B.4 Activity Diagrams

The activity diagrams model the dynamic operational workflows of MTBS across customers, theater managers, administrators, and the system web portal.

B.4.1 Customer Movie Booking & Payment Workflow

- Pre-conditions: Customer is registered and authenticated.
- Post-conditions: Ticket is booked, seats updated to BOOKED, payment verified, and digital receipt generated.

Execution Sequence:

1. Customer logs in and explores the active movie catalog.
2. Customer clicks 'Select Seats' for the chosen movie.
3. System displays the interactive seating map highlighting available vs booked seats.
4. Customer selects one or more available seats; client script computes real-time pricing.
5. Customer clicks 'Proceed to Checkout'; system executes transactional booking reservation.
6. Customer selects preferred payment method (UPI, Card, Net Banking) and submits payment.
7. System validates payment, issues digital ticket receipt, and records transaction.

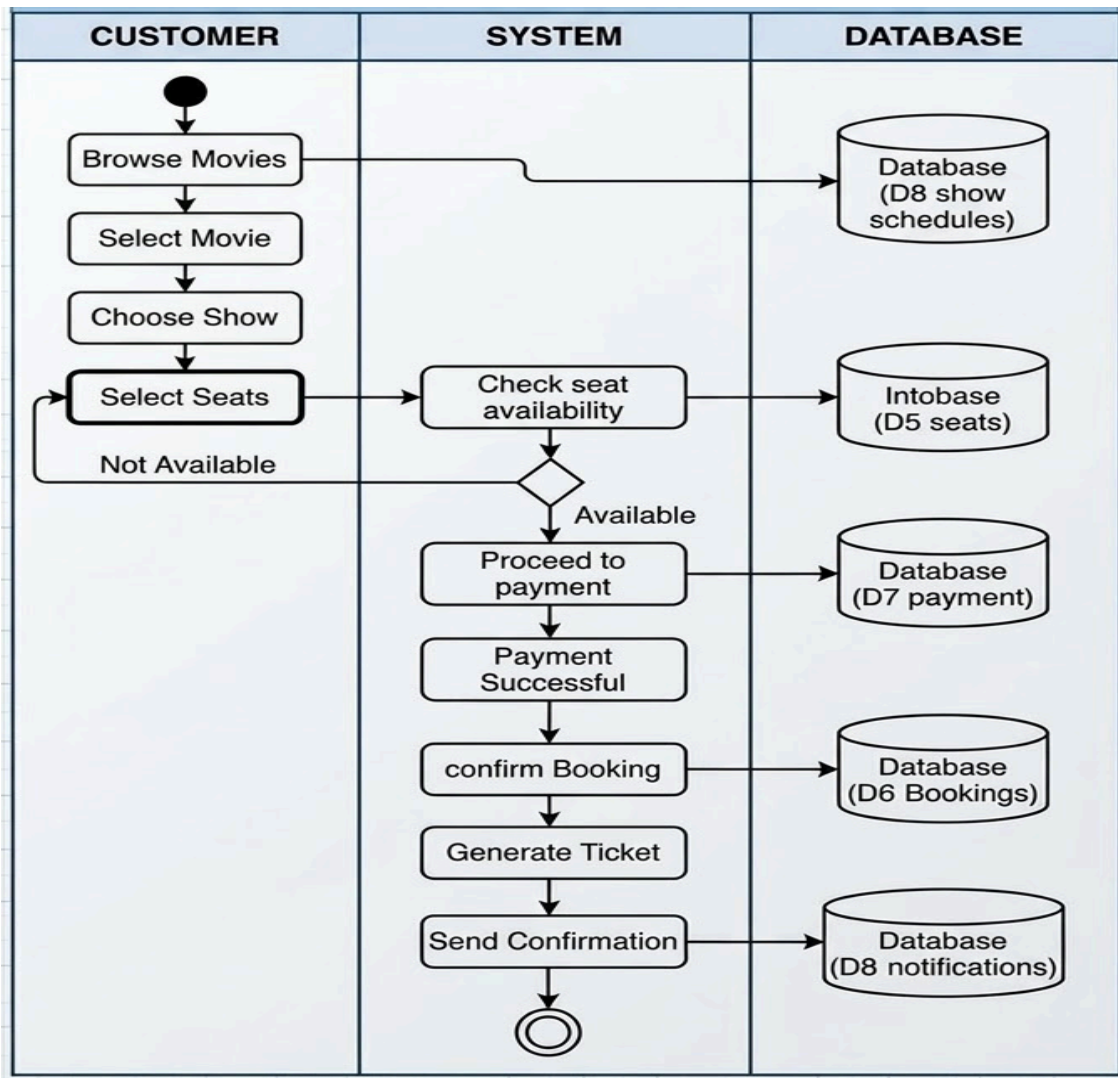


Figure B.2 Activity Diagram - Customers

B.4.2 Theater Manager Movie & Showtime Scheduling Workflow

- Pre-conditions: Theater Manager is authenticated.
- Post-conditions: New film title is registered and assigned screening slots.

Execution Sequence:

1. Manager logs into the Manager Control Center.
2. Manager inputs new movie details (title, genre, language, duration, description).
3. System inserts movie record and auto-schedules an initial showtime.
4. Manager navigates to 'Add Show', selects screening hall, date, time slot, and ticket price.
5. System validates slot parameters and updates the active cinema schedule.

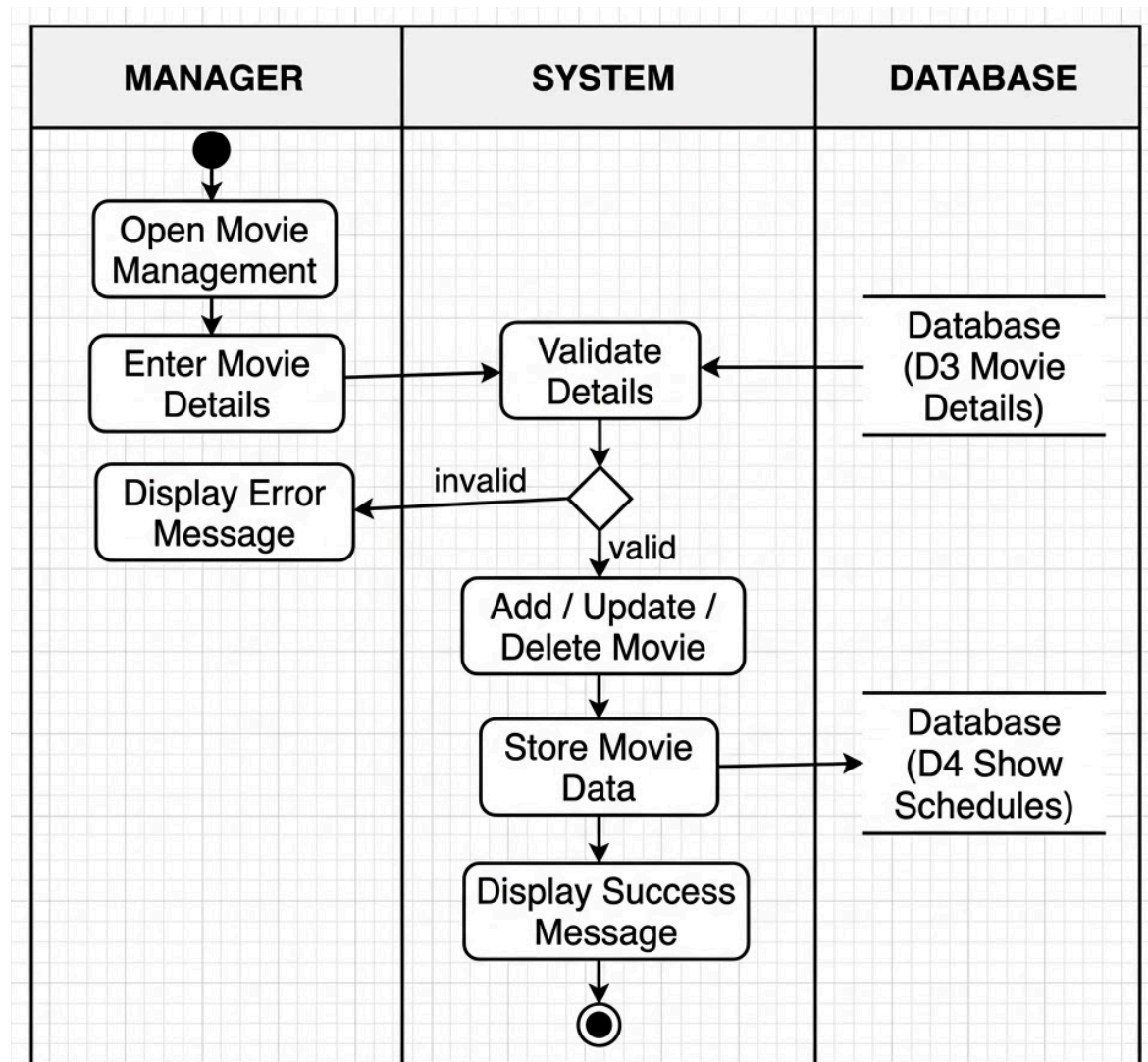


Figure B.3 Activity Diagram - Managers

B.5 Sequence Diagrams

B.5.1 Admin managing manager and customers

- Purpose: Illustrates object interactions during customer seat reservation and payment checkout.
- Participants: Customer, Web Interface, BookingServlet, PaymentServlet, MySQL Database.

Execution Flow:

1. Customer selects seats and clicks 'Proceed to Checkout'.
2. Web Interface issues HTTP POST /booking to BookingServlet.
3. BookingServlet verifies session and begins transaction with MySQL Database.
4. BookingServlet inserts booking record and updates seat status to BOOKED.
5. Database returns generated Booking ID; transaction commits.
6. BookingServlet redirects Web Interface to payment.jsp?bookingId=X.
7. Customer selects payment method and clicks 'Pay & Issue Ticket Now'.
8. PaymentServlet inserts payment record and redirects to ticket.jsp.

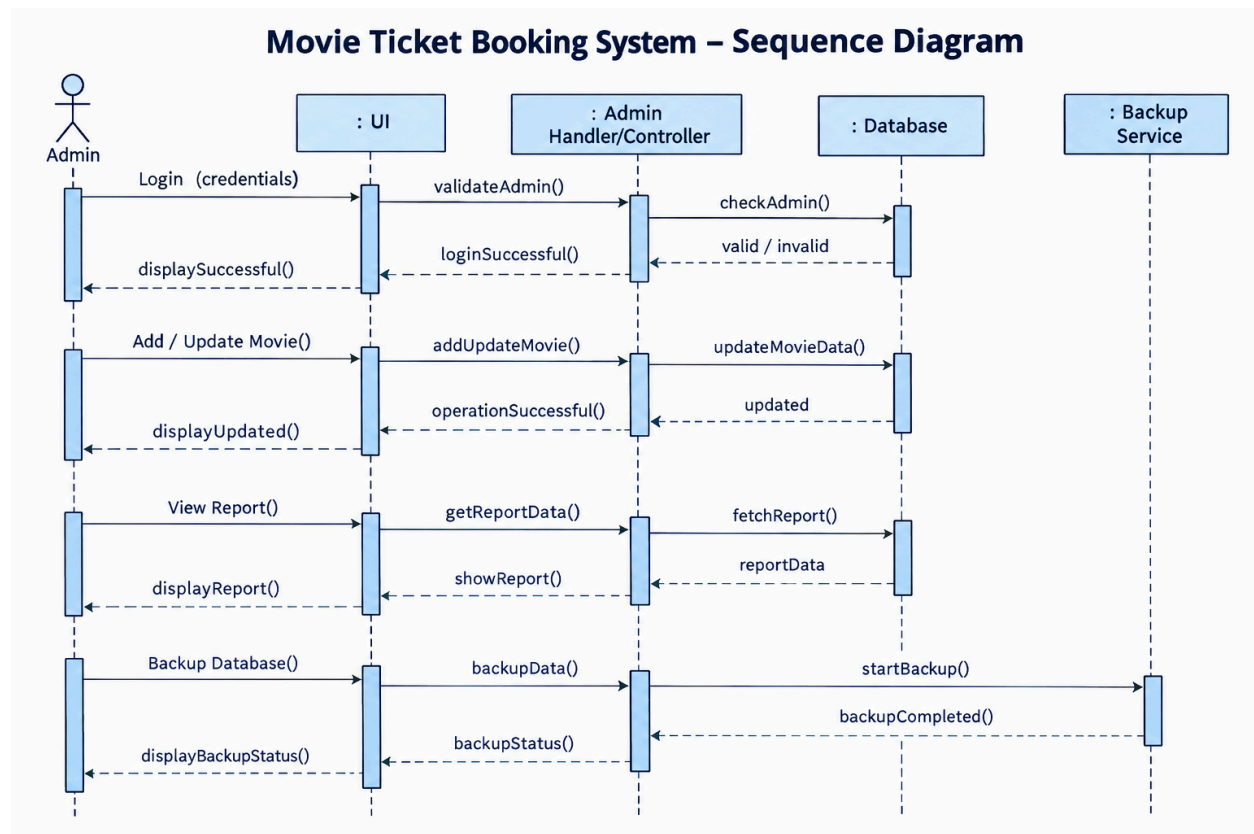


Figure B.4 Sequence Diagram - Admin

B.5.2 Theater Manager Showtime Scheduling Sequence

- Purpose: Illustrates message flow when Manager schedules a screening slot.
- Participants: Theater Manager, Web Interface, AddShowServlet, MySQL Database.

Execution Flow:

1. Manager selects movie, screen, date, time, and price on addShow.jsp.
2. Web Interface sends POST /addShow to AddShowServlet.
3. AddShowServlet validates parameters and queries shows table.
4. AddShowServlet executes INSERT into shows table.
5. Database confirms insertion; Servlet redirects Manager to manager.jsp?showAdded=1.

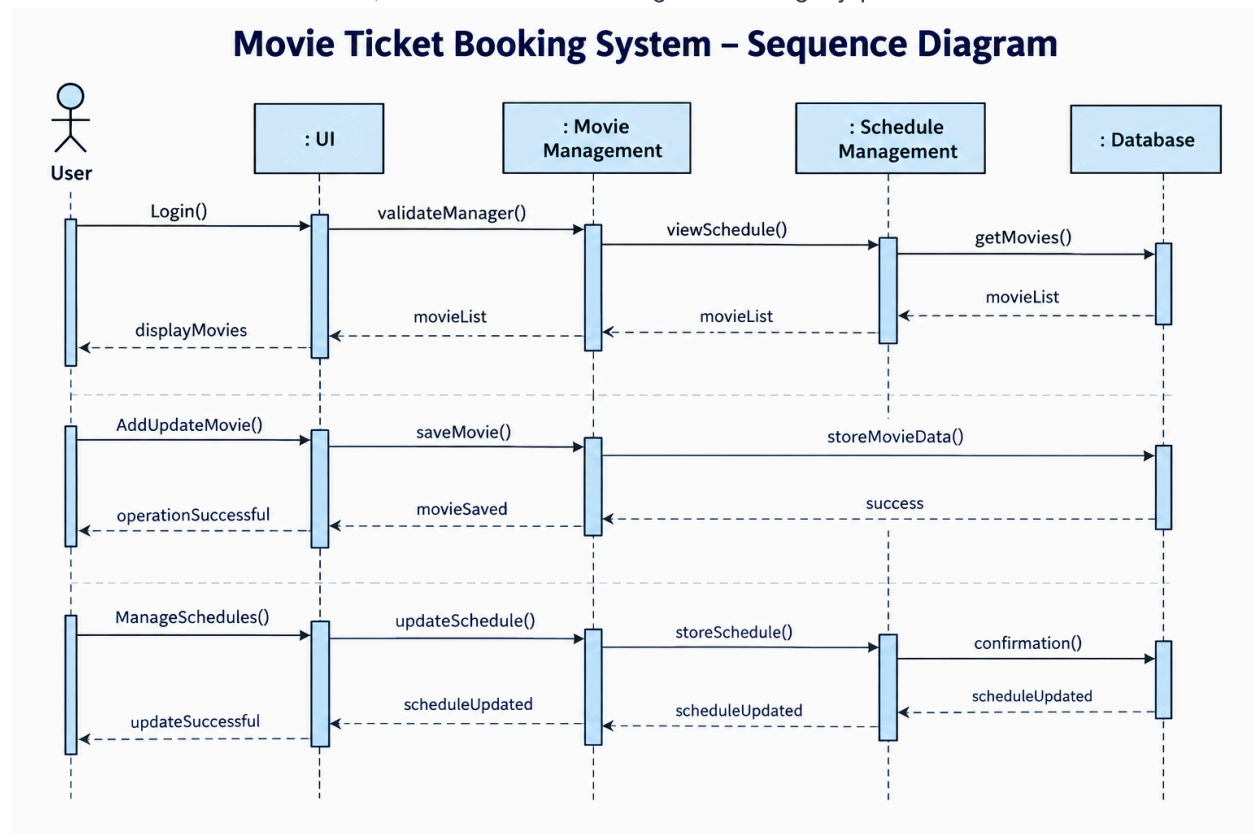


Figure B.5 Sequence Diagram - Manager

B.6 State Chart Diagram

B.6.1 Seat & Ticket Booking Lifecycle

The state chart models the states and transitions of cinema seats and ticket bookings in MTBS:

Lifecycle State	State Description & Behavioral Triggers
AVAILABLE	Seat is vacant and open for selection by any customer on seats.jsp.
LOCKED / SELECTED	Customer has toggled checkbox on seats.jsp; total price dynamically computed.
BOOKED	Booking transaction committed; seat status set to BOOKED; blocked for all other users.
CANCELLED	Customer cancelled booking; system atomically resets seat status to AVAILABLE.

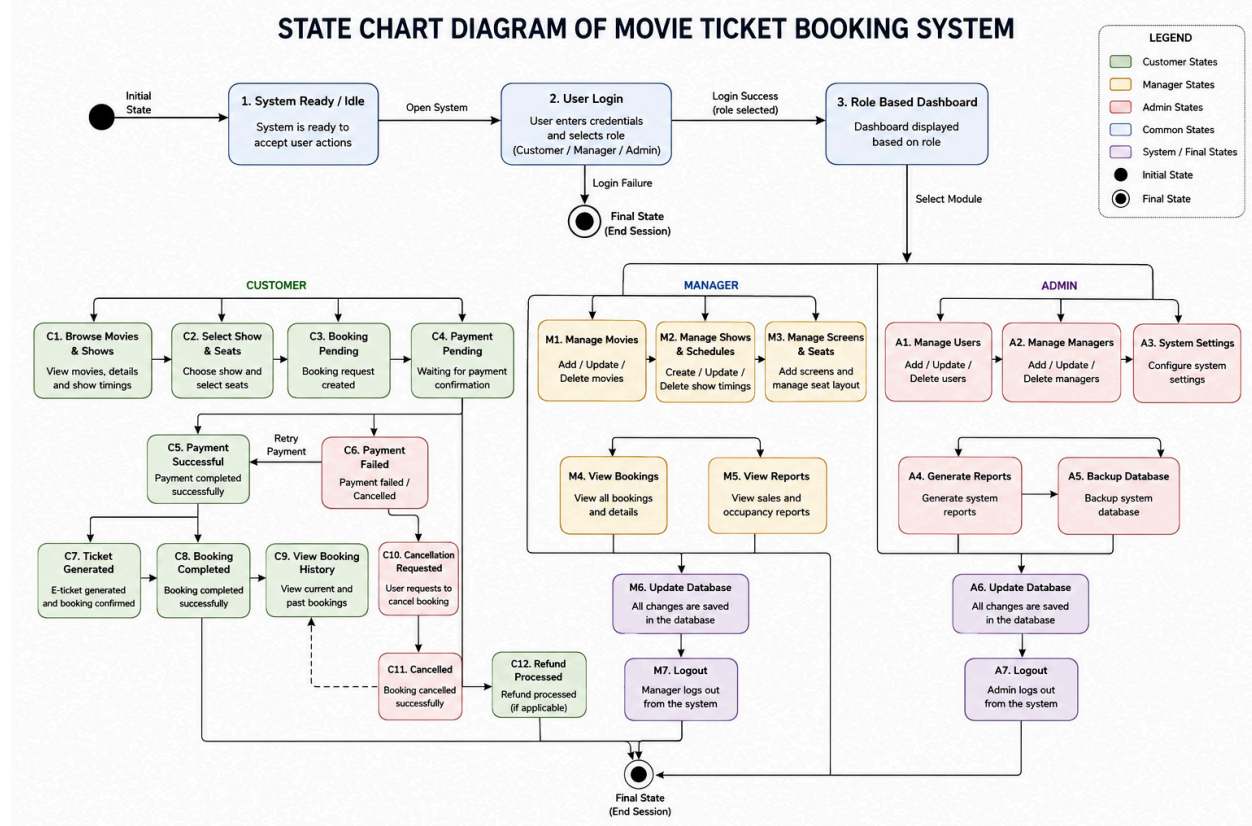


Figure B.6 State Chart Diagram

B.7 UML Component Diagram

B.7.1 Components of the System

The component diagram illustrates the modular architecture of MTBS:

Component Name	Functional Architectural Responsibility
Customer Portal	JSP interfaces providing movie browsing, interactive seat selection, payment, and tickets.
Manager Portal	JSP interfaces enabling movie cataloging, show scheduling, and sales reporting.
Admin Console	JSP interfaces providing user directory audit and manager provisioning.
Authentication Service	LoginServlet and session filters managing credential verification and RBAC.

Catalog & Scheduling Service	MovieServlet, AddMovieServlet, and AddShowServlet handling film and show data.
Booking Engine	BookingServlet executing atomic seat reservations and transaction rollbacks.
Payment Service	PaymentServlet recording payment transactions and issuing ticket references.
Database Tier	MySQL 8.0 database managing relational tables with referential integrity.

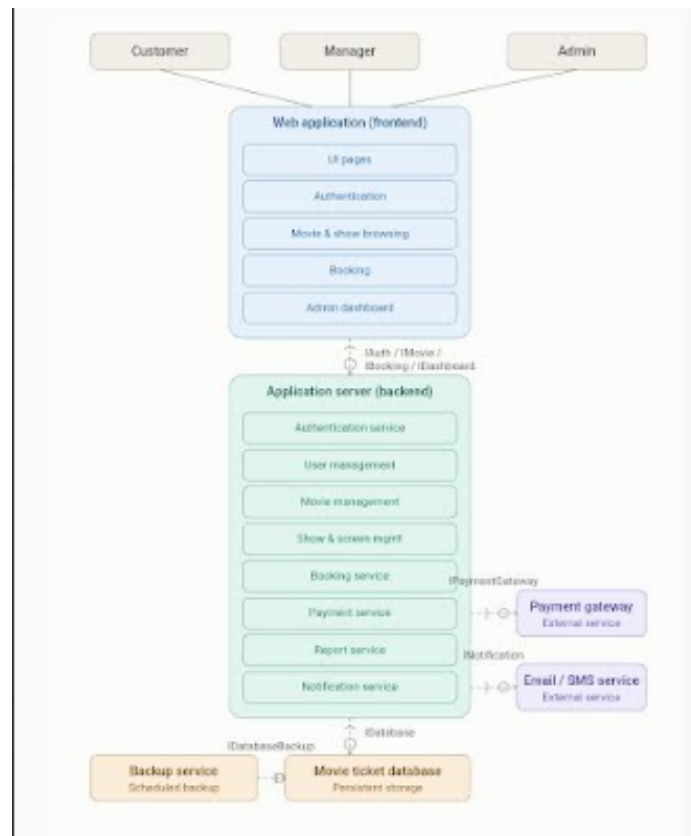


Figure B.7 UML Component Diagram

B.8 UML Deployment Diagram

B.8.1 Physical Deployment Architecture

The deployment diagram illustrates physical hardware nodes, execution environments, and communication protocols:

Deployment Node	Hardware Specification & Software Environment
Client Device Node	Standard desktop/mobile web browser rendering HTML5/CSS3 interface over HTTPS.
Web & App Server Node	Apache Tomcat 9.0 servlet engine hosting WAR archive and compiling JSP/Servlets.
Database Server Node	MySQL 8.0 Server managing movie_ticket_db storage and transaction logs.

Connection Link	Protocol & Port	Communication Description
Client Device to Web Server	HTTP / HTTPS (Port 8080 / 443)	Dispatches web pages, form submissions, and static CSS assets.
Application Server to DB Server	JDBC / TCP (Port 3306)	Executes SQL queries and transactional commits via MySQL Connector/J.

UML Deployment Diagram Movie Ticket Booking System

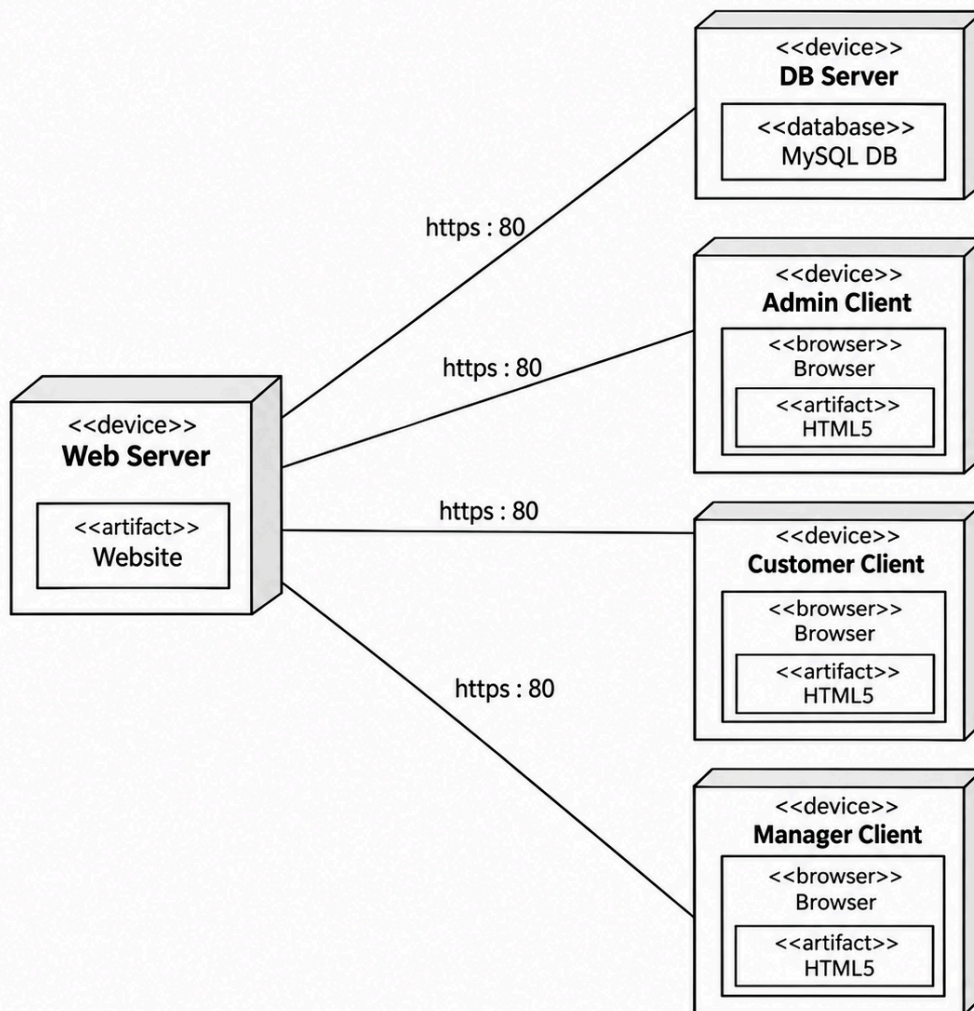


Figure B.8 UML Deployment Diagram

Appendix C:Source Code:

AddManagerServlet.java:

```
package com.moviebooking.controller;

import com.moviebooking.util.DBConnection;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

import java.sql.*;

@WebServlet("/addManager")

public class AddManagerServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,
        HttpServletResponse res)

        throws IOException {

        HttpSession session = req.getSession(false);

        String role = session != null ? (String)

            session.getAttribute("role") : null;

        // Security check: Only ADMIN can add managers

        if (!"ADMIN".equals(role)) {

            res.sendRedirect("login.jsp");

            return;

        }

        String name = req.getParameter("name");

        String email = req.getParameter("email");

        String password = req.getParameter("password");

        String phone = req.getParameter("phone");

        if (name == null || email == null || password == null)

        {

            res.sendRedirect("addManager.jsp?error=1");

            return;

        }

        String sql = "INSERT INTO users(name, email,

            password, phone, role) VALUES(?, ?, ?, ?, 'MANAGER')";

        try (Connection c =
```

```
DBConnection.getConnection();
```

```
    PreparedStatement p =
```

```
c.prepareStatement(sql) {
```

```
    p.setString(1, name);
```

```
    p.setString(2, email);
```

```
    p.setString(3, password);
```

```
    p.setString(4, phone != null ? phone : "");
```

```
    p.executeUpdate();
```

```
    res.sendRedirect("userDirectory.jsp?added=1");
```

```
    } catch (Exception e) {
```

```
        e.printStackTrace();
```

```
        res.sendRedirect("addManager.jsp?error=1");
```

```
    } } }
```

AddMovieServlet.java:

```
package com.moviebooking.controller;
```

```
import com.moviebooking.util.DBConnection;
```

```
import javax.servlet.annotation.WebServlet;
```

```
import javax.servlet.http.*;
```

```
import java.io.IOException;
```

```
import java.sql.*;
```

```
@WebServlet("/addMovie")
```

```
public class AddMovieServlet extends HttpServlet {
```

```

protected void doPost(HttpServletRequest req,
    HttpServletResponse res)

    throws IOException {

    String sql = "INSERT INTO movies(title, genre,
    language, duration, description) VALUES(?, ?, ?, ?, ?)";

    try (Connection c =
    DBConnection.getConnection());

        PreparedStatement p =
    c.prepareStatement(sql,
    Statement.RETURN_GENERATED_KEYS)) {

        p.setString(1, req.getParameter("title"));

        p.setString(2, req.getParameter("genre"));

        p.setString(3, req.getParameter("language"));

        p.setInt(4,
    Integer.parseInt(req.getParameter("duration")));

        p.setString(5, req.getParameter("description"));

        p.executeUpdate();

        ResultSet keys = p.getGeneratedKeys();

        if (keys.next()) {

            int newMovieId = keys.getInt(1);

            PreparedStatement psShow =
    c.prepareStatement(

                "INSERT INTO shows(movie_id, screen_id,
    show_date, show_time, price) VALUES(?, 1,

```

```

CURDATE(), '18:00:00', 150.00)");

        psShow.setInt(1, newMovieId);

        psShow.executeUpdate();

    }

    res.sendRedirect("movies");

} catch (Exception e) {

    e.printStackTrace();

    res.sendRedirect("addMovie.jsp?error=1");

} } }

```

AddShowServlet.java

```

package com.moviebooking.controller;

import com.moviebooking.util.DBConnection;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

import java.sql.*;

@WebServlet("/addShow")

public class AddShowServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,

```

```

HttpServletResponse res)

    throws IOException {

    String movieIdStr = req.getParameter("movieId");

    String screenIdStr = req.getParameter("screen");

    String showDate =
req.getParameter("show_date");

    String showTime =
req.getParameter("show_time");

    String priceStr = req.getParameter("price");

    try (Connection c =
DBConnection.getConnection();

        PreparedStatement p = c.prepareStatement(

            "INSERT INTO shows(movie_id, screen_id,
show_date, show_time, price) VALUES(?, ?, ?, ?, ?)") {

        p.setInt(1, Integer.parseInt(movieIdStr));

        p.setInt(2, Integer.parseInt(screenIdStr));

        p.setDate(3, Date.valueOf(showDate));

        p.setTime(4, Time.valueOf(showTime.length()
== 5 ? showTime + ".00" : showTime));

        p.setDouble(5, Double.parseDouble(priceStr));

        p.executeUpdate();

        res.sendRedirect("manager.jsp?showAdded=1");

    } catch (Exception e) {

        e.printStackTrace();

        res.sendRedirect("addShow.jsp?error=1");

    }

}

}

```

BookingServlet.java

```

package com.moviebooking.controller;

import com.moviebooking.util.DBConnection;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

import java.sql.*;

@WebServlet("/booking")

public class BookingServlet extends HttpServlet {

```

```

protected void doPost(HttpServletRequest req,
HttpServletResponse res)

    throws IOException {

    HttpSession s = req.getSession(false);

    Integer userId = s == null ? null : (Integer)
s.getAttribute("userId");

    String[] seats = req.getParameterValues("seats");

    int showId = 1;

    try {

        String showIdStr = req.getParameter("showId");

        if (showIdStr != null &&
!showIdStr.trim().isEmpty()) {

            showId = Integer.parseInt(showIdStr.trim());

        }

    } catch (Exception ignored) {}

    if (userId == null) {

        res.sendRedirect("login.jsp");

        return;

    }

    if (seats == null || seats.length == 0) {

        res.sendRedirect("seats.jsp?showId=" + showId

+ "&error=1");

        return;

    }

    Connection c = null;

    try {

        c = DBConnection.getConnection();

        if (c == null) {

            res.sendRedirect("seats.jsp?showId=" +
showId + "&error=failed");

            return;

        }

        c.setAutoCommit(false);

        // 1. Ensure show exists in shows table to
prevent foreign key violation

        PreparedStatement psCheckShow =
c.prepareStatement("SELECT show_id, price FROM
shows WHERE show_id = ?");

        psCheckShow.setInt(1, showId);

        ResultSet rsCheckShow =

```

```
psCheckShow.executeQuery();                                ?, CURDATE(), '18:00:00', 150.00));

    double price = 150.0;                                insShow.setInt(1, showId);

    if (rsCheckShow.next()) {                             insShow.setInt(2, movieId);

        double p = rsCheckShow.getDouble("price");        insShow.setInt(3, screenId);

        if (p > 0) price = p;                             insShow.executeUpdate();

    } else {                                              }

    int movieId = 1;

    Statement st = c.createStatement();                   double total = price * seats.length;

    ResultSet rsM = st.executeQuery("SELECT
movie_id FROM movies LIMIT 1");

    if (rsM.next()) {                                     // 2. Insert booking record

        movieId = rsM.getInt(1);                           PreparedStatement p = c.prepareStatement(

    }                                                       "INSERT INTO bookings(user_id, show_id,

    int screenId = 1;

    ResultSet rsS = st.executeQuery("SELECT
screen_id FROM screens LIMIT 1");

    if (rsS.next()) {

        screenId = rsS.getInt(1);

    }

    PreparedStatement insShow =
c.prepareStatement(

        "INSERT INTO shows(show_id, movie_id,
screen_id, show_date, show_time, price) VALUES(?, ?,
```

```
total_amount, status) VALUES(?, ?, ?, 'CONFIRMED')",
```

```
Statement.RETURN_GENERATED_KEYS);
```

```
p.setInt(1, userId);
```

```
p.setInt(2, showId);
```

```
p.setDouble(3, total);
```

```
p.executeUpdate();
```

```
ResultSet k = p.getGeneratedKeys();
```

```
if (!k.next()) {
```

```
    c.rollback();
```

```
    res.sendRedirect("seats.jsp?showId=" +  
showId + "&error=failed");
```

```
    return;
```

```
}
```

```
int bookingId = k.getInt(1);
```

```
// 3. Mark selected seats as BOOKED and  
record in booking_seats
```

```
PreparedStatement seatCheck =  
c.prepareStatement("SELECT status FROM seats  
WHERE seat_id = ?");
```

```
PreparedStatement seatUpdate =  
c.prepareStatement(
```

```
    "UPDATE seats SET status = 'BOOKED'
```

```
WHERE seat_id = ? AND status = 'AVAILABLE'");
```

```
PreparedStatement bs = c.prepareStatement(
```

```
    "INSERT INTO booking_seats(booking_id,  
seat_id) VALUES(?, ?)");
```

```
for (String x : seats) {
```

```
    int seatId = Integer.parseInt(x);
```

```
    seatCheck.setInt(1, seatId);
```

```
    ResultSet rsSeat =  
seatCheck.executeQuery();
```

```
    if (rsSeat.next()) {
```

```
        String status = rsSeat.getString("status");
```

```
        if ("BOOKED".equalsIgnoreCase(status)) {
```

```
            c.rollback();
```

```
            res.sendRedirect("seats.jsp?showId=" +  
showId + "&error=booked");
```

```
            return;
```

```
        }
```

```
        seatUpdate.setInt(1, seatId);
```

```
        int updatedRows =
```

```
seatUpdate.executeUpdate();

        if (updatedRows == 0) {

            c.rollback();

            res.sendRedirect("seats.jsp?showId=" +
showId + "&error=booked");

            return;

        }

    } else {

        PreparedStatement insSeat =
c.prepareStatement(

            "INSERT INTO seats(seat_id, screen_id,
seat_number, status) VALUES(?, 1, ?, 'BOOKED')");

        insSeat.setInt(1, seatId);

        insSeat.setString(2, "S" + seatId);

        insSeat.executeUpdate();

    }

    bs.setInt(1, bookingId);

    bs.setInt(2, seatId);

    bs.executeUpdate();

}

c.commit();

res.sendRedirect("payment.jsp?bookingId=" +
bookingId);

    } catch (Exception e) {

        e.printStackTrace();

        if (c != null) {

            try { c.rollback(); } catch (Exception ignored) {}

        }

        res.sendRedirect("seats.jsp?showId=" + showId
+ "&error=failed");

    } finally {

        if (c != null) {

            try { c.setAutoCommit(true); c.close(); }


```

```
catch (Exception ignored) {}  
  
    }  
  
    }  
  
    }  
  
}
```

```
import javax.servlet.annotation.WebServlet;  
  
import javax.servlet.http.*;  
  
import java.io.IOException;  
  
import java.sql.*;  
  
  
@WebServlet("/cancelBooking")  
  
public class CancelBookingServlet extends  
HttpServlet {  
  
    protected void doGet(HttpServletRequest  
req, HttpServletResponse res)  
  
        throws IOException {  
  
        int id=Integer.parseInt(req.getParameter("id"));  
  
        try(Connection  
c=DBConnection.getConnection()){  
  
            c.setAutoCommit(false);  
  
            PreparedStatement p=c.prepareStatement(  
  
                "UPDATE bookings SET status='CANCELLED'  
WHERE booking_id=?");  
  
            p.setInt(1,id); p.executeUpdate();  
  
            PreparedStatement q=c.prepareStatement(  
  
                "UPDATE seats s JOIN booking_seats bs ON  
s.seat_id=bs.seat_id SET s.status='AVAILABLE'  
WHERE bs.booking_id=?");  
  
            q.setInt(1,id); q.executeUpdate();  
  
            c.commit();
```

CancelBookingServlet.java

```
package com.moviebooking.controller;  
  
  
  
import com.moviebooking.util.DBConnection;
```

```
res.sendRedirect("bookingHistory.jsp");

    } catch(Exception e){ e.printStackTrace();
res.sendRedirect("bookingHistory.jsp?error=1"); }

}

}
```

DeleteMovieServlet.java

```
package com.moviebooking.controller;

import com.moviebooking.dao.MovieDAO;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

@WebServlet("/deleteMovie")

public class DeleteMovieServlet extends HttpServlet {

    protected void doGet(HttpServletRequest
req,HttpServletResponse res)

        throws IOException {

            new
MovieDAO().deleteMovie(Integer.parseInt(req.getPara
```

```
meter("id")));

        res.sendRedirect("movies");

    }

}
```

DeleteUserServlet.java

```
package com.moviebooking.controller;

import com.moviebooking.util.DBConnection;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

import java.sql.*;

@WebServlet("/deleteUser")

public class DeleteUserServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req,
HttpServletResponse res)

        throws IOException {

            HttpSession session = req.getSession(false);

            String role = session != null ? (String)
session.getAttribute("role") : null;

            Integer currentUserId = session != null ? (Integer)
```

```
session.getAttribute("userId") : null;                                ;

                                                                    return;

// Security check: Only ADMIN can delete user accounts             }

if (!"ADMIN".equals(role)) {

                                                                    try (Connection c =
                                                                    DBConnection.getConnection());

                                                                    PreparedStatement p =
                                                                    c.prepareStatement("DELETE FROM users WHERE
                                                                    user_id = ?") {

                                                                    p.setInt(1, targetId);

                                                                    p.executeUpdate();

String idParam = req.getParameter("id");

                                                                    if (idParam == null) {

                                                                    res.sendRedirect("userDirectory.jsp");

                                                                    return;

                                                                    }

                                                                    int targetId = Integer.parseInt(idParam);

                                                                    // Prevent admin from self-deleting

                                                                    if (currentUserId != null && currentUserId ==
                                                                    targetId) {

                                                                    res.sendRedirect("userDirectory.jsp?error=self_delete")
```

```

res.sendRedirect("userDirectory.jsp?deleted=1");

    } catch (Exception e) {

        e.printStackTrace();

        res.sendRedirect("userDirectory.jsp?error=1");

    } } }

```

LoginServlet.java

```

package com.moviebooking.controller;

import com.moviebooking.util.DBConnection;

import javax.servlet.ServletException;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

import java.sql.*;

@WebServlet("/login")

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest

```

```

req,HttpServletResponse res)

    throws ServletException,IOException {

    String email=req.getParameter("email");

    String password=req.getParameter("password");

    try(Connection c=DBConnection.getConnection();

        PreparedStatement p=c.prepareStatement(

            "SELECT * FROM users WHERE email=? AND

            password=?")) {

        p.setString(1,email); p.setString(2,password);

        ResultSet r=p.executeQuery();

        if(r.next()){

            HttpSession s=req.getSession();

            s.setAttribute("userId",r.getInt("user_id"));

            s.setAttribute("name",r.getString("name"));

            s.setAttribute("role",r.getString("role"));

            String role=r.getString("role");

            if("ADMIN".equals(role))

                res.sendRedirect("admin.jsp");

            else if("MANAGER".equals(role))

                res.sendRedirect("manager.jsp");

            else res.sendRedirect("customer.jsp");

        } else res.sendRedirect("login.jsp?error=1");

    } catch(Exception e){ e.printStackTrace();

```

```
res.sendRedirect("login.jsp?error=1"); }

}}
```

LogoutServlet.java

```
package com.moviebooking.controller;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

@WebServlet("/logout")

public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request
```

```
req, HttpServletResponse res)

    throws IOException {

        HttpSession s=req.getSession(false);

        if(s!=null) s.invalidate();

        res.sendRedirect("login.jsp");

    } }
```

MovieServlet.java

```
package com.moviebooking.controller;

import com.moviebooking.dao.MovieDAO;

import javax.servlet.*;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.*;

import java.io.IOException;

@WebServlet("/movies")

public class MovieServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request
req, HttpServletResponse res)

    throws ServletException, IOException {

        req.setAttribute("movies", new
MovieDAO().getAllMovies());

        req.getRequestDispatcher("movies.jsp").forward(req, r
```

```
es);
```

```
    } }
```

PaymentServlet.java

```
package com.moviebooking.controller;
```

```
import com.moviebooking.util.DBConnection;
```

```
import javax.servlet.annotation.WebServlet;
```

```
import javax.servlet.http.*;
```

```
import java.io.IOException;
```

```
import java.sql.*;
```

```
@WebServlet("/payment")
```

```
public class PaymentServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
```

```
    {
```

```
        int
```

```
        bookingId=Integer.parseInt(req.getParameter("bookingId"));
```

```
        String method=req.getParameter("method");
```

```
        try(Connection c=DBConnection.getConnection();
```

```
            PreparedStatement p=c.prepareStatement(
```

```
                "SELECT total_amount FROM bookings
```

```
                WHERE booking_id=?)") {
```

```
                    p.setInt(1,bookingId); ResultSet  
                    r=p.executeQuery();
```

```
                    double amount=0; if(r.next())  
                    amount=r.getDouble(1);
```

```
                    PreparedStatement q=c.prepareStatement(
```

```
                        "INSERT INTO  
                        payments(booking_id,amount,payment_method,payment_status) VALUES(?,?,?, 'SUCCESS')");
```

```
                        q.setInt(1,bookingId); q.setDouble(2,amount);  
                        q.setString(3,method);
```

```
                        q.executeUpdate();
```

```
                    res.sendRedirect("ticket.jsp?bookingId="+bookingId);
```

```
                } catch(Exception e){ e.printStackTrace();
```

```
res.sendRedirect("payment.jsp?error=1"); }  
  
    } }  
  
RegisterServlet.java  
  
package com.moviebooking.controller;  
  
import com.moviebooking.util.DBConnection;  
  
import javax.servlet.annotation.WebServlet;  
  
import javax.servlet.http.*;  
  
import java.io.IOException;  
  
import java.sql.*;  
  
@WebServlet("/register")  
  
public class RegisterServlet extends HttpServlet {  
  
    protected void doPost(HttpServletRequest request  
    req, HttpServletResponse res)  
  
        throws IOException {  
  
        String sql="INSERT INTO  
        users(name,email,password,phone,role)  
        VALUES(?,?,?,?,?)";  
  
        try(Connection c=DBConnection.getConnection();  
  
            PreparedStatement p=c.prepareStatement(sql))  
  
            {  
  
                p.setString(1,request.getParameter("name"));  
  
                p.setString(2,request.getParameter("email"));  
  
                p.setString(3,request.getParameter("password"));  
  
                p.setString(4,request.getParameter("phone"));  
  
                p.setString(5,"CUSTOMER");  
  
                p.executeUpdate();  
  
                res.sendRedirect("login.jsp?registered=1");  
  
            } catch(Exception e){ e.printStackTrace();  
  
            }  
  
        }  
  
    }  
  
}
```

```
res.sendRedirect("register.jsp?error=1"); }

} }
```

MovieDAO.java

```
package com.moviebooking.dao;
```

```
import com.moviebooking.model.Movie;
```

```
import com.moviebooking.util.DBConnection;
```

```
import java.sql.*;
```

```
import java.util.*;
```

```
public class MovieDAO {
```

```
    public List<Movie> getAllMovies() {
```

```
        List<Movie> list = new ArrayList<>();
```

```
        String sql = "SELECT * FROM movies";
```

```
        try(Connection c=DBConnection.getConnection();
```

```
            PreparedStatement p=c.prepareStatement(sql);
```

```
            ResultSet r=p.executeQuery()) {
```

```
            while(r.next()) {
```

```
                Movie m=new Movie();
```

```
                m.setMovieId(r.getInt("movie_id"));
```

```
                m.setTitle(r.getString("title"));
```

```
                m.setGenre(r.getString("genre"));
```

```
                m.setLanguage(r.getString("language"));
```

```
                m.setDuration(r.getInt("duration"));
```

```
                m.setDescription(r.getString("description"));
```

```
                list.add(m);
```

```
            }
```

```
        } catch(Exception e){ e.printStackTrace(); }
```

```
        return list;
```

```
    }
```

```
    public boolean deleteMovie(int id) {
```

```
        try(Connection c=DBConnection.getConnection();
```

```
            PreparedStatement p=c.prepareStatement(
```

```
                "DELETE FROM movies WHERE movie_id=?")
```

```
        {
```

```
            p.setInt(1,id);
```

```
            return p.executeUpdate()>0;
```

```
        } catch(Exception e){ e.printStackTrace(); return
```

```
false; }
```

```
    } }
```

Movie.java

```
package com.moviebooking.model;
```

```
public class Movie {
```

```
    private int moviId, duration;
```

```
    private String title, genre, language, description;
```

```
    public int getMoviId(){ return moviId; }
```

```
    public void setMoviId(int v){ moviId=v; }
```

```
    public int getDuration(){ return duration; }
```

```
    public void setDuration(int v){ duration=v; }
```

```
    public String getTitle(){ return title; }
```

```
    public void setTitle(String v){ title=v; }
```

```
    public String getGenre(){ return genre; }
```

```
    public void setGenre(String v){ genre=v; }
```

```
    public String getLanguage(){ return language; }
```

```
    public void setLanguage(String v){ language=v; }
```

```
    public String getDescription(){ return description; }
```

```
    public void setDescription(String v){ description=v; }
```

```
}
```

User.java

```
package com.moviebooking.model;
```

```
public class User {
```

```
    private int userId;
```

```
    private String name, email, password, phone, role;
```

```
    public int getUserId(){ return userId; }
```

```
    public void setUserId(int v){ userId=v; }
```

```
    public String getName(){ return name; }
```

```
    public void setName(String v){ name=v; }
```

```
    public String getEmail(){ return email; }
```

```
    public void setEmail(String v){ email=v; }
```

```
    public String getPassword(){ return password; }
```

```
    public void setPassword(String v){ password=v; }
```

```
    public String getPhone(){ return phone; }
```

```
    public void setPhone(String v){ phone=v; }
```

```
    public String getRole(){ return role; }
```

```
    public void setRole(String v){ role=v; }
```

```
}
```

DBConnection.java

package com.moviebooking.util;	charset=UTF-8" pageEncoding="UTF-8"%>
import java.sql.Connection;	<!DOCTYPE html>
import java.sql.DriverManager;	<html lang="en">
public class DBConnection {	<head>
private static final String URL = "jdbc:mysql://127.0.0.1:3306/movie_ticket_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";	<meta charset="UTF-8">
	<meta name="viewport" content="width=device-width, initial-scale=1.0">
private static final String USER = "root";	<title>Login CinePass Movie Booking</title>
private static final String PASSWORD = "root";	<link rel="stylesheet" href="css/style.css">
public static Connection getConnection() {	</head>
try {	<body>
Class.forName("com.mysql.cj.jdbc.Driver");	<div class="auth-wrapper">
return DriverManager.getConnection(URL, USER, PASSWORD);	<div class="auth-card">
} catch (Exception e) {	<div class="auth-header">
try {	<div class="brand-icon">🎬</div>
return DriverManager.getConnection("jdbc:mysql://localhost:3306/movie_ticket_db", USER, PASSWORD);	Welcome Back
} catch (Exception ex) {	<p class="page-subtitle">Sign in to your CinePass account</p>
ex.printStackTrace();	</div>
return null;	<% if("1".equals(request.getParameter("error")))%>
} } }	<div class="alert alert-danger">
	⚠ Invalid email or password.

Login.jsp

```
<%@ page language="java" contentType="text/html;
```

```
Please try again.</span>      </div>

    <% } %>      <%
if("1".equals(request.getParameter("registered"))) { %>

    <div class="alert alert-success">

        <span>🎉 Registration successful! Please
log in below.</span>

    </div>

    <% } %>

    <form action="login" method="post">

        <div class="form-group">

            <label class="form-label">Email
Address</label>

            <input type="email" name="email"
placeholder="name@example.com" required
autocomplete="email">

        </div>

        <div class="form-group">

            <label class="form-
label">Password</label>

            <input type="password" name="password"
placeholder="....." required>

        </div>

        <button type="submit" class="btn btn-
primary btn-block">Sign In</button>

    </form>

    <div style="margin-top: 1.5rem; font-size:
```

```
0.9rem; color: var(--text-muted);">

        Don't have an account? <a href="register.jsp"
style="font-weight: 600;">Create Account</a>

    </div>    </div>    </div></body></html>
```

Register.jsp

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-
width, initial-scale=1.0">

    <title>Create Account | CinePass</title>

    <link rel="stylesheet" href="css/style.css">

</head>

<body>

    <div class="auth-wrapper">

        <div class="auth-card">

            <div class="auth-header">

                <div class="brand-icon">🎬</div>

                <h2>Create Account</h2>

                <p class="page-subtitle">Join CinePass for
```

seamless movie tickets</p>

</div>

```
<% if("1".equals(request.getParameter("error")))
{ %>
```

```
<div class="alert alert-danger">
```

```
<span>⚠ Registration failed. Email may
already exist.</span>
```

```
</div> <% } %>
```

```
<form action="register" method="post">
```

```
<div class="form-group">
```

```
<label class="form-label">Full
Name</label>
```

```
<input type="text" name="name"
placeholder="John Doe" required> </div>
```

```
<div class="form-group">
```

```
<label class="form-label">Email
Address</label>
```

```
<input type="email" name="email"
placeholder="john@example.com" required>
```

```
</div>
```

```
<div class="form-group">
```

```
<label class="form-
label">Password</label>
```

```
<input type="password" name="password"
```

```
placeholder="*****" required>
```

```
</div>
```

```
<div class="form-group">
```

```
<label class="form-label">Phone
Number</label>
```

```
<input type="text" name="phone"
placeholder="+1 (555) 000-0000" required>
```

```
</div>
```

```
<button type="submit" class="btn btn-
primary btn-block">Register Account</button>
```

```
</form>
```

```
<div style="margin-top: 1.5rem; font-size:
0.9rem; color: var(--text-muted);">
```

```
Already registered? <a href="login.jsp"
style="font-weight: 600;">Sign In</a>
```

```
</div> </div> </div></body></html>
```

Admin.jsp

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
```

```
<%@ page session="true" %>
```

```
<%
```

```
String userName = (String)
```

session.getAttribute("name");	0.25rem;">Admin
String role = (String) session.getAttribute("role");	</div>
if(!"ADMIN".equals(role)) {	<ul class="nav-links">
response.sendRedirect("login.jsp");	Dashboard
return; }	User Directory
%>	
<!DOCTYPE html>	Add Manager
<html lang="en">	🛡️ Admin (<%=userName%>)
<head>	
<meta charset="UTF-8">	Logout
<meta name="viewport" content="width=device-width, initial-scale=1.0">	</nav>
<title>Admin Dashboard CinePass</title>	<main class="app-container">
<link rel="stylesheet" href="css/style.css">	<div class="page-header">
</head>	<div class="page-title">
<body>	System Administrator Dashboard
<nav class="navbar">	<p class="page-subtitle">Manage manager accounts, user permissions, and platform configuration.</p> </div>
<div class="navbar-brand">	
<div class="brand-icon">⚡</div>	<a href="addManager.jsp" class="btn btn-
CinePass <span style="font-size: 0.75rem; color: var(--gold); text-transform: uppercase; letter-spacing: 0.1em; margin-left:	

primary">+ Add New Manager

</div>

<div class="dashboard-grid">

<div class="stat-card">

<div class="stat-icon">👤</div>

<div class="stat-info">

<h4>User System</h4>

<div class="stat-value">Accounts</div>

</div>

</div>

<div class="stat-card">

<div class="stat-icon">👤</div>

<div class="stat-info">

<h4>Theater Managers</h4>

<div class="stat-value">Staff</div>

</div>

</div>

<div class="stat-card">

<div class="stat-icon">🛡️</div>

<div class="stat-info">

<h4>Security & Roles</h4>

<div class="stat-value">Control</div>

</div>

</div>

</div>

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)); gap: 1.5rem;">

<div class="glass-card">

<h3 style="margin-bottom: 0.5rem;">👤 User Directory</h3>

<p style="color: var(--text-muted); font-size: 0.95rem; margin-bottom: 1.5rem;">View all user accounts, remove outdated manager or customer accounts, and audit roles.</p>

Open User Directory →

</div>

<div class="glass-card">

<h3 style="margin-bottom: 0.5rem;">+ Add New Manager</h3>

<p style="color: var(--text-muted); font-size: 0.95rem; margin-bottom: 1.5rem;">Provision a new theater manager account with full access to add movies and showtimes.</p>

Create Manager Account →

</div>

</div>

</main>

<footer class="footer">

<p>© 2026 CinePass Enterprise Administration Panel.</p>

</footer></body></html>

Manager.jsp

<%@ page language="java" contentType="text/html;">

charset=UTF-8" pageEncoding="UTF-8"%>	COUNT(*) FROM movies");
<%@ page import="java.sql.*" %>	if(r1.next()) totalMovies = r1.getInt(1);
<%@ page import="com.moviebooking.util.DBConnection" %>	ResultSet r2 = s.executeQuery("SELECT COUNT(*) FROM shows");
<%@ page session="true" %>	if(r2.next()) totalShows = r2.getInt(1);
<%	ResultSet r3 = s.executeQuery("SELECT COUNT(*), COALESCE(SUM(total_amount), 0) FROM bookings WHERE status='CONFIRMED'");
String userName = (String) session.getAttribute("name");	if(r3.next()) {
String role = (String) session.getAttribute("role");	totalBookings = r3.getInt(1);
if(!"MANAGER".equals(role)) {	totalRevenue = r3.getDouble(2);
response.sendRedirect("login.jsp");	} }
return;	} catch(Exception e) {
}	e.printStackTrace();
int totalMovies = 0;	}
int totalShows = 0;	%>
int totalBookings = 0;	<!DOCTYPE html>
double totalRevenue = 0.0;	<html lang="en">
try(Connection c = DBConnection.getConnection()) {	<head>
if(c != null) {	<meta charset="UTF-8">
try(Statement s = c.createStatement()) {	<meta name="viewport" content="width=device-
ResultSet r1 = s.executeQuery("SELECT	

width, initial-scale=1.0">	0.85rem;">Logout
<title>Manager Portal CinePass</title>	
<link rel="stylesheet" href="css/style.css">	</nav>
</head>	<main class="app-container">
<body>	<div class="page-header">
<nav class="navbar">	<div class="page-title">
<div class="navbar-brand">	Theater Manager Control Center
<div class="brand-icon"> </div>	<p class="page-subtitle">Welcome back,
CinePass Manager	<%=userName%>. Oversee movie schedules, ticket bookings, and sales revenue.</p>
</div>	</div>
<ul class="nav-links">	<div style="display: flex; gap: 0.75rem;">
Dashboard	 View Sales Report
Movies Catalog	 Add Movie
Add Movie	</div>
Add Show	</div>
Sales Report	<div class="dashboard-grid">
 <%=userName%>	<div class="stat-card">
<a href="logout" class="btn btn-secondary" style="padding: 0.4rem 0.9rem; font-size:	<div class="stat-icon"></div>
	<div class="stat-info">
	Movie Catalog
	<div class="stat-value"><%=totalMovies%>

Active</div>

</div>

</div>

<div class="stat-card">

<div class="stat-icon"></div>

<div class="stat-info">

<h4>Showtimes</h4>

<div class="stat-value"><%=totalShows%>
Scheduled</div>

</div>

</div>

<div class="stat-card">

<div class="stat-icon"></div>

<div class="stat-info">

<h4>Total Sales</h4>

<div class="stat-value" style="color: var(--
accent);"><%=String.format("%.0f",

totalRevenue)%></div>

</div>

</div>

<div class="stat-card">

<div class="stat-icon"></div>

<div class="stat-info">

<h4>Tickets Sold</h4>

<div class="stat-
value"><%=totalBookings%></div>

</div>

</div>

</div>

<div style="display: grid; grid-template-columns:
repeat(auto-fit, minmax(280px, 1fr)); gap: 1.5rem;">

<div class="glass-card">

<h3 style="margin-bottom: 0.5rem;">
Add Movie</h3>

<p style="color: var(--text-muted); font-size:
0.9rem; margin-bottom: 1.25rem;">Register a new
movie title into the cinema system with duration and
description.</p>

<a href="addMovie.jsp" class="btn btn-
secondary btn-block">Add Movie Form &arr;

</div>

<div class="glass-card">

<h3 style="margin-bottom: 0.5rem;">

Manage Movies

`<p style="color: var(--text-muted); font-size: 0.9rem; margin-bottom: 1.25rem;">View catalog, check details, or remove outdated movies from circulation.</p>`

`<a href="movies" class="btn btn-secondary btn-block">View Movie List →</a>`

`</div>`

`<div class="glass-card">`

`<h3 style="margin-bottom: 0.5rem;"> Schedule Show</h3>`

`<p style="color: var(--text-muted); font-size: 0.9rem; margin-bottom: 1.25rem;">Create new showtime slots, assign theater halls, and set screen pricing.</p>`

`<a href="addShow.jsp" class="btn btn-secondary btn-block">Create Show Slot →</a>`

`</div>`

`<div class="glass-card">`

`<h3 style="margin-bottom: 0.5rem;"> Sales & Revenue Reports</h3>`

`<p style="color: var(--text-muted); font-size: 0.9rem; margin-bottom: 1.25rem;">Analyze total ticket revenue, view recent booking transactions, and audit payment methods.</p>`

`<a href="reports.jsp" class="btn btn-primary`

`btn-block">Open Sales Report →</a>`

`</div>`

`</div>`

`</main>`

`<footer class="footer">`

`<p>© 2026 CinePass Management Console. All rights reserved.</p>`

`</footer></body></html>`

Customer.jsp

`<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>`

`<%@ page session="true" %>`

`<%`

`String userName = (String)`

session.getAttribute("name");	width, initial-scale=1.0">
if(userName == null) {	<title>Customer Dashboard CinePass</title>
response.sendRedirect("login.jsp");	<link rel="stylesheet" href="css/style.css">
return;	</head>
}	<body>
%>	<nav class="navbar">
<!DOCTYPE html>	<div class="navbar-brand">
<html lang="en">	<div class="brand-icon">🎬</div>
<head>	CinePass
<meta charset="UTF-8">	</div>
<meta name="viewport" content="width=device-	<ul class="nav-links">
	Dashboard
	Browse Movies
	My Bookings
	👤 <%=userName%>
	<a href="logout" class="btn btn-secondary" style="padding: 0.4rem 0.9rem; font-size:

```
0.85rem;">Logout</a></li>
```

```
</ul>
```

```
</nav>
```

```
<main class="app-container">
```

```
<div class="page-header">
```

```
<div class="page-title">
```

```
<h1>Welcome back, <span class="gradient-  
title"><%=userName%></span> 🖐️</h1>
```

```
<p class="page-subtitle">Ready to  
experience cinema on the big screen today?</p>
```

```
</div>
```

```
<a href="movies" class="btn btn-primary">🍿  
Browse Now Showing</a>
```

```
</div>
```

```
<div class="dashboard-grid">
```

```
<div class="stat-card">
```

```
<div class="stat-icon">📺 </div>
```

```
<div class="stat-info">
```

```
<h4>Available Movies</h4>
```

```
<div class="stat-value">Now
```

```
Showing</div>
```

```
</div>
```

```
</div>
```

```
<div class="stat-card">
```

```
<div class="stat-icon">🎟️ </div>
```

```
<div class="stat-info">
```

```
<h4>Active Bookings</h4>
```

```
<div class="stat-value">Instant  
Ticket</div>
```

```
</div>
```

```
</div>
```

```
<div class="stat-card">
```

```
<div class="stat-icon">★</div>
```

```
<div class="stat-info">
```

```
<h4>Member Status</h4>
```

```
<div class="stat-value">VIP Pass</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div style="display: grid; grid-template-columns:  
repeat(auto-fit, minmax(300px, 1fr)); gap: 1.5rem;  
margin-top: 1.5rem;">
```

```
<div class="glass-card" style="display: flex; flex-
```

direction: column; justify-content: space-between;">

<div>

<h3 style="font-size: 1.3rem; margin-bottom: 0.5rem;">Browse Latest Movies</h3>

<p style="color: var(--text-muted); font-size: 0.95rem; margin-bottom: 1.5rem;">Explore upcoming releases, check showtimes, and select your favorite seats.</p>

</div>

Explore Movies &arr;

</div>

<div class="glass-card" style="display: flex; flex-direction: column; justify-content: space-between;">

<div>

<h3 style="font-size: 1.3rem; margin-bottom: 0.5rem;">My Booking History</h3>

<p style="color: var(--text-muted); font-size: 0.95rem; margin-bottom: 1.5rem;">View your confirmed tickets, digital receipts, or manage your bookings.</p>

</div>

View History &arr;

</div> </div> </main>

<footer class="footer">

<p>© 2026 CinePass Movie Ticket Booking

System. All rights reserved.</p>

</footer></body></html>

addMovie.jsp

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<%@ page session="true" %>

<%

String userName = (String)
session.getAttribute("name");

String role = (String) session.getAttribute("role");

%>

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-

width, initial-scale=1.0">	0.85rem;">Logout
<title>Add Movie CinePass</title>	<% } %> </nav>
<link rel="stylesheet" href="css/style.css">	<main class="app-container" style="max-width: 680px;">
</head>	<div class="page-header">
<body>	<div class="page-title">
<nav class="navbar">	Add New Movie
<div class="navbar-brand">	<p class="page-subtitle">Fill in details to add a movie title to the active theater schedule.</p>
<div class="brand-icon"> </div>	</div>
CinePass	← Back to Movies
</div>	</div>
<ul class="nav-links">	<% if("1".equals(request.getParameter("error")))
Manager Dashboard	{ %>
Movies Catalog	<div class="alert alert-danger">
Add Movie	 Failed to add movie. Please check your inputs and try again.
Add Show	</div> <% } %>
Sales Report	<div class="glass-card">
<% if(userName != null) { %>	<form action="addMovie" method="post">
 <%=userName%>	<div class="form-group">
	<label class="form-label">Movie Title</label>
<a href="logout" class="btn btn-secondary" style="padding: 0.4rem 0.9rem; font-size:	<input type="text" name="title"

placeholder="e.g. Inception 2" required>

</div>

<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">

<div class="form-group">

<label class="form-label">Genre</label>

<input type="text" name="genre" placeholder="Action / Sci-Fi" required>

</div>

<div class="form-group">

<label class="form-label">Language</label>

<input type="text" name="language" placeholder="English / Hindi" required>

</div> </div>

<div class="form-group">

<label class="form-label">Duration (in Minutes)</label>

<input type="number" name="duration" placeholder="148" min="1" required>

</div>

<div class="form-group">

<label class="form-label">Synopsis / Description</label>

<textarea name="description" placeholder="Enter movie storyline summary, cast

details..." required></textarea>

</div>

<button type="submit" class="btn btn-primary btn-block">+ Add Movie to Catalog</button>

</form> </div> </main>

<footer class="footer">

<p>© 2026 CinePass Management Console.</p>

</footer></body></html>

addShow.jsp

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<%@ page import="java.sql.*" %>

<%@ page import="com.moviebooking.util.DBConnection" %>

<%@ page session="true" %>

<%

String userName = (String) session.getAttribute("name");

String role = (String) session.getAttribute("role");

if(!"MANAGER".equals(role) &&

```

! "ADMIN".equals(role)) {

    response.sendRedirect("login.jsp");

    return;

}

%>

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-
width, initial-scale=1.0">

    <title>Add Show | CinePass</title>

    <link rel="stylesheet" href="css/style.css">

</head>

<body>

    <nav class="navbar">

        <div class="navbar-brand">

            <div class="brand-icon">📅 </div>

            <span>CinePass</span>

        </div>

        <ul class="nav-links">

            <li><a href="manager.jsp" class="nav-
link">Manager Dashboard</a></li>

            <li><a href="movies" class="nav-link">Movies
Catalog</a></li>

            <li><a href="addMovie.jsp" class="nav-
link">Add Movie</a></li>

            <li><a href="addShow.jsp" class="nav-link
active">Add Show</a></li>

            <li><a href="reports.jsp" class="nav-link">Sales
Report</a></li>

            <% if(userName != null) { %>

                <li><span class="user-badge">🔧
<%=userName%></span></li>

                <li><a href="logout" class="btn btn-
secondary" style="padding: 0.4rem 0.9rem; font-size:
0.85rem;">Logout</a></li>

            <% } %>        </ul>    </nav>

            <main class="app-container" style="max-width:
680px;">

                <div class="page-header">

                    <div class="page-title">

                        <h1>Schedule New Showtime</h1>

                        <p class="page-subtitle">Assign a movie to a
screen hall and pick date & time slots.</p>

                    </div>

                    <a href="manager.jsp" class="btn btn-
secondary">⏪ Back to Dashboard</a>

                </div>

                <div>

                    <a href="manager.jsp" class="nav-

                    <% if("1".equals(request.getParameter("error"))>

```

```

{ %>
    <div class="alert alert-danger">
        <span>⚠ Failed to schedule showtime.
Please check inputs and try again.</span>
    </div>
<% } %>
<div class="glass-card">
    <form action="addShow" method="post">
        <div class="form-group">
            <label class="form-label">Select
Movie</label>
            <select name="movieId" required>
                <%
                    try (Connection c =
DBConnection.getConnection();
                        Statement s = c.createStatement();
                            ResultSet r =
s.executeQuery("SELECT movie_id, title FROM movies
ORDER BY title ASC")) {
                                while (r.next()) {
                                    %>
                                        <option
value="<%=r.getInt("movie_id")%>"><%=r.getString("titl
e")%></option>
                                } catch (Exception e) {
                                    e.printStackTrace();
                                }
                            %>
                        </select>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Screen
Hall</label>
                        <select name="screen">
                            <option value="1">Screen 1 (IMAX
4K)</option>
                            <option value="2">Screen 2 (Dolby
Atmos)</option>
                            <option value="3">Screen 3 (VIP
Lounge)</option>
                        </select>
                    </div>
                    <div style="display: grid; grid-template-
columns: 1fr 1fr; gap: 1rem;">
                        <div class="form-group">
                            <label class="form-label">Show

```

```
Date</label>                                     Console.</p>

    <input type="date" name="show_date"
required>                                     </footer>

    </div>                                     </body>

    <div class="form-group">                   </html>

        <label class="form-label">Show
Time</label>

        <input type="time" name="show_time"
required>

        </div>        </div>

    <div class="form-group">

        <label class="form-label">Base Ticket
Price (₹)</label>

        <input type="number" name="price"
value="150" min="50" step="10" required>

        </div>

        <button type="submit" class="btn btn-
primary btn-block">Schedule Showtime</button>

    </form>

</div>

</main>

<footer class="footer">

    <p>&copy; 2026 CinePass Management
```

Output:

Customer Account Registration:

The screenshot displays a 'Create Account' registration form on a dark-themed background. At the top, there is a small icon of a person with a red 'X' and the title 'Create Account'. Below the title is the subtitle 'Join CinePass for seamless movie ticketing'. The form contains four input fields: 'FULL NAME' with the value 'Rahul Sharma', 'EMAIL ADDRESS' with 'rahul.sharma@gmail.com', 'PHONE NUMBER' with '9876543210', and 'PASSWORD' with masked characters. A prominent blue 'Create Account' button is positioned below the password field. At the bottom of the form, a link reads 'Already registered? Sign In Here'. A footer label at the bottom right of the screen identifies the process as 'CinePass Operation 02: Customer Account Registration'.

Figure C.1 Customer Account Registration

Login & Role Authentication:

The screenshot shows a 'Welcome Back' login form on a dark-themed background. At the top, there is a small icon of a person and the title 'Welcome Back'. Below the title is the subtitle 'Sign in to your CinePass account'. The form includes two input fields: 'EMAIL ADDRESS' with the value 'rahul.customer@gmail.com' and 'PASSWORD' with masked characters. A prominent blue 'Sign In' button is located below the password field. Below the button, a link reads 'Don't have an account? Create Account'. At the bottom of the form, the text 'Multi-Role Authentication Portal (Customer / Manager / Admin)' is displayed. A footer label at the bottom right of the screen identifies the process as 'CinePass Operation 01: Login & Role Authentication'.

Figure C.2 Login & Role Authentication

Customer Dashboard:

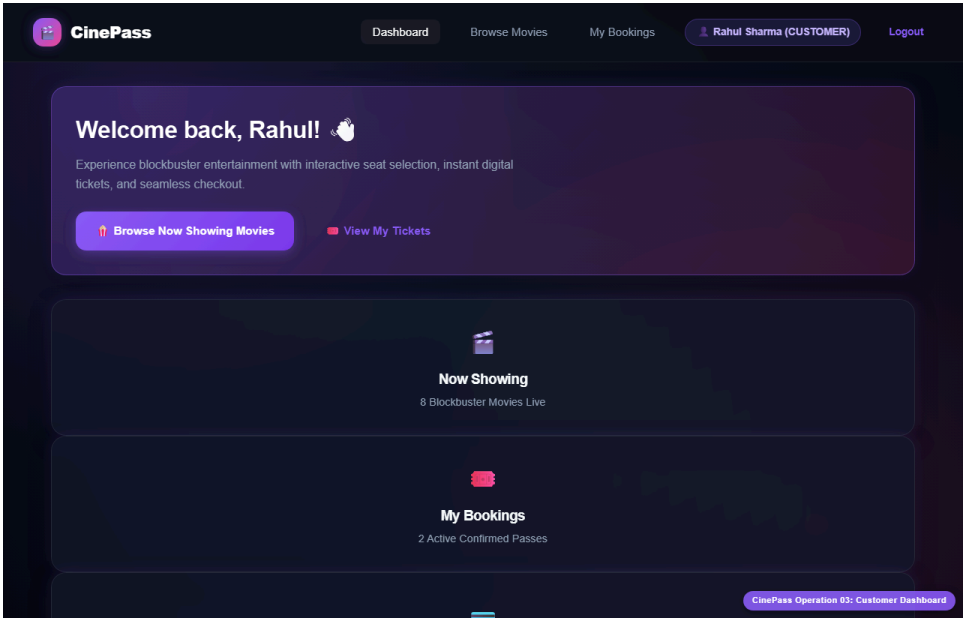


Figure C.3 Customer Dashboard

Movies Catalog & Screening Details:

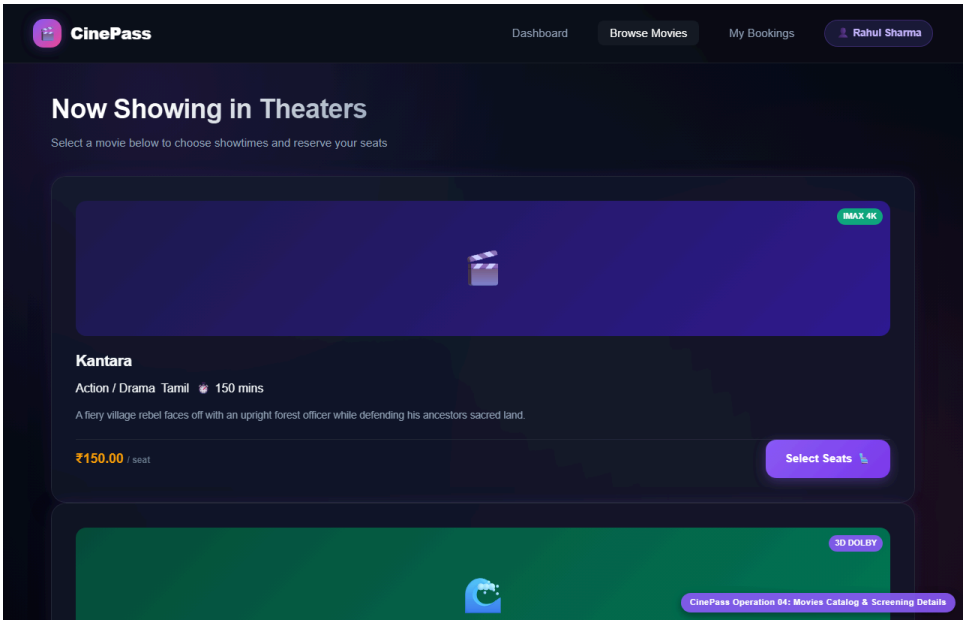


Figure C.4 Movies Catalog & Screening Details

Simulated Payment Checkout:

The image shows a simulated payment checkout interface for CinePass. The interface is dark-themed with a central white card containing the payment details. At the top left of the card is the CinePass logo. The title 'Payment Checkout Gateway' is prominently displayed, followed by the instruction 'Complete payment to issue your digital movie tickets'. Below this is an 'Order Booking Summary' section with a table of booking details. The table lists the movie title 'Kantara', screen hall 'Screen 1 IMAX', reserved seats 'A1, A2', and show date & time 'Today, 18:00 PM'. The total payable amount is ₹300.00. Under the summary, there is a 'SELECT PAYMENT METHOD' section with two options: 'UPI / GPay / PhonePe' (selected) and 'Credit / Debit Card'. Below the payment methods is a 'VIRTUAL PAYMENT ADDRESS (UPI ID)' field with the value 'rahul.sharma@upi'. At the bottom of the card is a large orange button labeled 'Pay ₹300.00 & Issue Ticket Now'. A footer note at the bottom right of the interface reads 'CinePass Operation 96: Simulated Payment Checkout'.

Order Booking Summary	
Movie Title: Kantara	Screen Hall: Screen 1 IMAX
Reserved Seats: A1, A2	Show Date & Time: Today, 18:00 PM
Total Payable Amount: ₹300.00	

SELECT PAYMENT METHOD

☒ UPI / GPay / PhonePe ☐ Credit / Debit Card

VIRTUAL PAYMENT ADDRESS (UPI ID)

rahul.sharma@upi

Pay ₹300.00 & Issue Ticket Now

CinePass Operation 96: Simulated Payment Checkout

Figure C.5 Simulated Payment Checkout

Interactive Graphical Seat Selection:

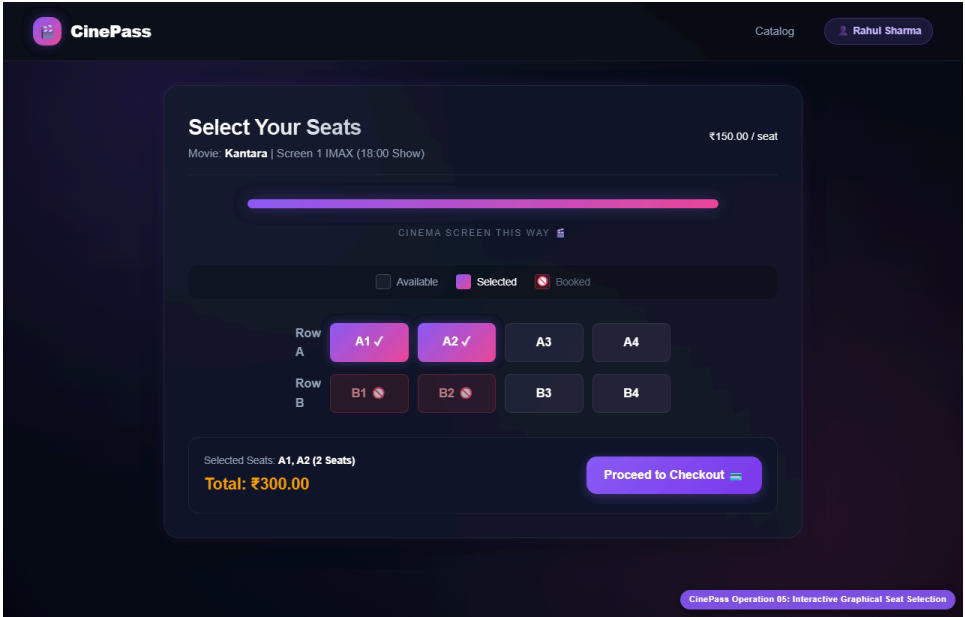


Figure C.6 Interactive Graphical Seat Selection

Digital Ticket Pass & Barcode:

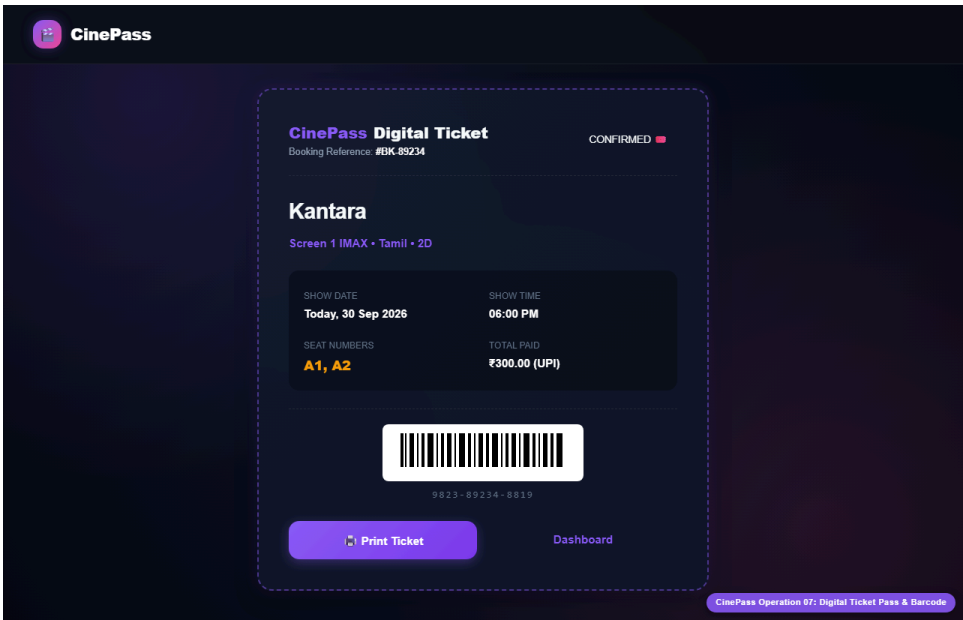


Figure C.7 Digital Ticket Pass & Barcode

Customer Booking History & Self-Service Cancellation:

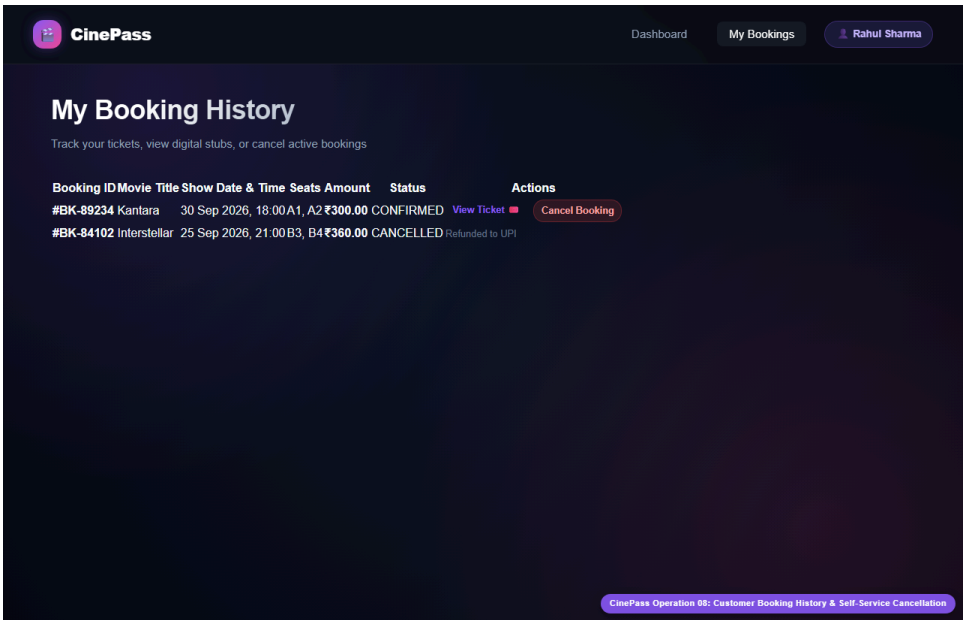


Figure C.8 Customer Booking History & Self-Service Cancellation

Theater Manager Control Center:

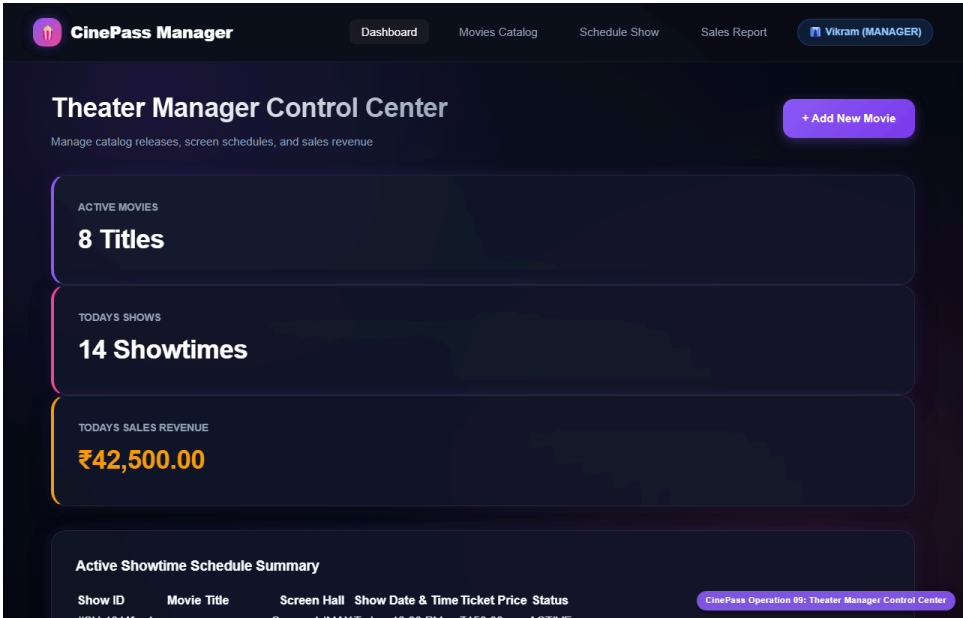


Figure C.9 Theater Manager Control Center

Add New Movie Form:

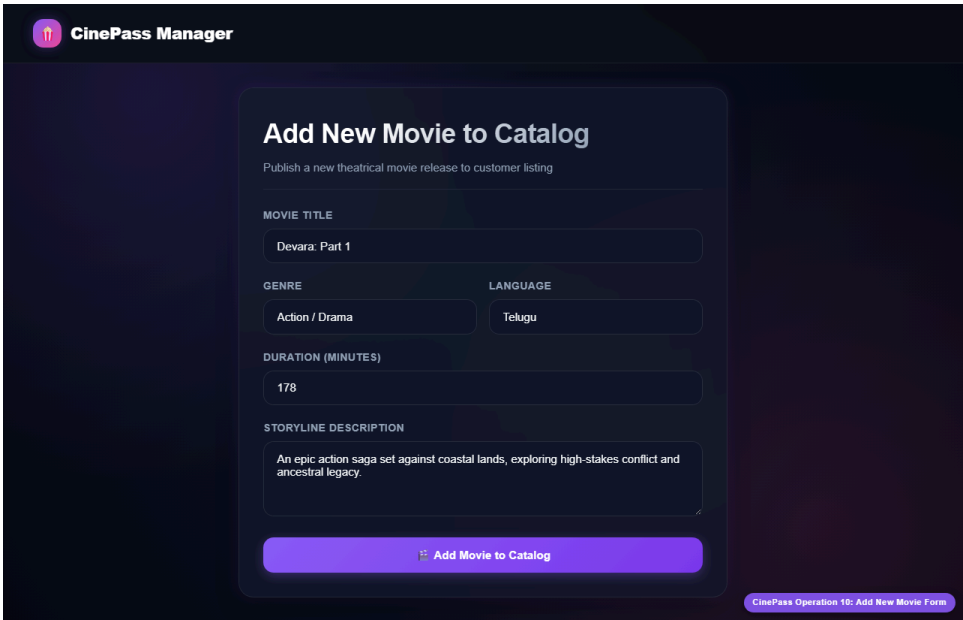


Figure C.10 Add New Movie Form

Manage Movies Catalog:

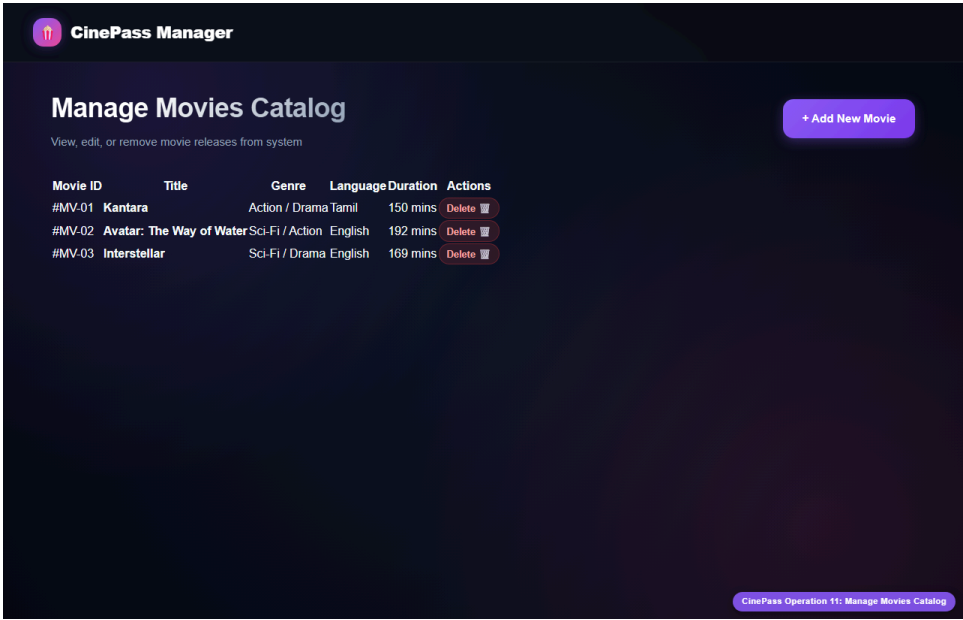


Figure C.11 Manage Movies Catalog

Schedule Showtime & Hall Allocation:

CinePass Manager

Schedule Showtime & Screen Slot

Assign movies to screen halls, showtimes, and ticket pricing

SELECT MOVIE
Kantara (Action / Drama - 150 mins)

SELECT SCREEN HALL
Screen IMAX (100 Capacity)

SHOW DATE
01-10-2026

SHOW TIME
18:00

BASE TICKET PRICE (INR ₹)
150.00

Schedule Showtime Slot

CinePass Operation 12: Schedule Showtime & Hall Allocation

Figure C.12 Schedule Showtime & Hall Allocation

E-Resources:

<https://github.com/nbarath1809-dot/MTBS.git>

Appendix D: To Be Determined List

The following items are scheduled for evaluation in upcoming project release cycles:

#	TBD Enhancement Item	Responsible Party	Target Timeline
1	Production payment gateway integration (Razorpay / Stripe) replacing simulated checkout.	Technical Lead	Q3 2026
2	Automated SMS / WhatsApp ticket delivery via Twilio API upon confirmed booking.	Integration Eng.	Q3 2026
3	Dynamic ticket pricing rules based on weekend vs weekday demand curves.	Product Manager	Q4 2026
4	Integration with physical cinema turnstile QR scanners for admission validation.	Operations Lead	Q4 2026

5	Mobile native application development for iOS and Android platforms.	Mobile Dev Team	Q1 2027
---	--	-----------------	---------